

MECH9325 Acoustics Toolkit

Full Backup: Formulas + MATLAB Source Code

Every calculator in the AcousticsApp toolkit (63 in total): its formula, a use-case note, and its **complete MATLAB source code**, verbatim from the repository. If getAIO, the app, or your internet connection ever fails, everything needed to recreate a working calculator by hand is in this document -- see the setup guide on the next page.

Reference constant	Value
p_ref (sound pressure)	2e-5 Pa (20 uPa, air)
W_ref (sound power)	1e-12 W
I_ref (sound intensity)	1e-12 W/m ²
rho*c (air, default)	415 rayls (1.21 kg/m ³ x 343 m/s)
c (air, default)	343 m/s
Underwater p_ref	1e-6 Pa (1 uPa); typical rho*c(water) = 1.5e6 rayls (1000 x 1500)

How To Use This As A Backup

There are two situations where you'd reach for this document instead of just running **getAIO** as normal.

Situation A -- the app is broken/buggy but you have internet

You don't need this PDF at all: keep using the toolkit normally. Report the bug and it can be fixed centrally, then **getAIO** gives everyone the fix. This PDF is specifically for when the toolkit itself is unreachable.

Situation B -- no internet, no USB copy, nothing but this PDF and MATLAB

Recreate whichever calculator(s) you need by hand. Every code block in this document is a complete, self-contained MATLAB function -- copy it out exactly as printed.

- Step 1: In MATLAB, create a new empty folder anywhere, e.g. C:\Acoustics (or ~/Acoustics on Mac/Linux). This will be your working folder.
- Step 2: Inside it, create a folder named EXACTLY +acoustics (the leading + is required -- it is what makes MATLAB treat the folder as a "package", so functions inside it are called as acoustics.functionName(...)).
- Step 3: For each formula you need, copy its code block from this PDF into a NEW script file in MATLAB (File > New > Script), then Save As with the EXACT filename shown above the code block (e.g. splPressure.m) INSIDE the +acoustics folder.
- Step 4: ALWAYS also copy in the three 'Support Files' at the end of this document (constants.m, weightingTable.m, weightingValue.m) -- many calculators depend on them internally, even ones that don't look like they would.
- Step 5: In MATLAB, set your Current Folder to C:\Acoustics (the folder that CONTAINS +acoustics -- not the +acoustics folder itself).
- Step 6: Call the function from the Command Window as acoustics.functionName(arguments) -- exactly like the worked example printed under each function's docstring.

Fastest safe option: if you're not sure which functions you'll need, just copy EVERY code block in this PDF into the +acoustics folder up front (about 65 small files) -- a few calculators call others internally, so this guarantees nothing is missing.

Worked example

```
>> cd C:\Acoustics | % the folder that CONTAINS +acoustics
>> R = acoustics.combineLevels([80 80 74]);
>> R.total
ans =
    83.4429
```

Two tools have no standalone file

"Leq time-varying level & percentile LN" and "Stats: sort values into terms" compute their results with logic written directly inside AcousticsApp.m rather than a separate package function. For these two only, this document includes a clean, independently-tested standalone rewrite (leqTimeVarying.m and sortValuesIntoTerms.m) that reproduces the app's results exactly and needs no +acoustics package at all -- save it anywhere and call it directly.

Contents

Unit Converter (1 calculators)
Levels: Basic Conversions & Point-Source Relations (7 calculators)
Combine & Subtract Sources (7 calculators)
Waves (6 calculators)
Distance Propagation (3 calculators)
Room Acoustics (7 calculators)
Sound Power Measurement (5 calculators)
Duct / Pipe (2 calculators)
Weighting & Band Workbench (2 calculators)
Time-Averaged Levels (Leq / SEL / Dose) (6 calculators)
Hearing Protection (2 calculators)
Loudness (2 calculators)
Speech & Community Noise (2 calculators)
Statistics (2 calculators)
Sound Insulation (Transmission Loss) (7 calculators)
Mufflers / Silencers (3 calculators)
Support Files (required by several calculators above)

Unit Converter

Unit converter (prefixes & acoustic units)

Convert any acoustic quantity between metric-prefixed or named units (Pa, W, Hz, dB refs, etc).

Generic metric prefixes on Pa, W, Hz, s, m, g, V, rayl:

f=1e-15 p=1e-12 n=1e-9 u=1e-6 m=1e-3 c=1e-2 d=1e-1 da=1e1 h=1e2 k=1e3 M=1e6 G=1e9 T=1e12

Also: bar/atm/psi/mmHg (pressure), in/ft/mile (length), min/hr/day (time), rad/s/rpm (frequency), m²/L/m³ (area/volume), km/h/knot (speed), kg/t (mass), C/K/F (temp, affine).

Source file: `acoustics/convertUnit.m`

```
function R = convertUnit(value, fromUnit, toUnit)
%CONVERTUNIT Convert acoustic quantities between units and metric prefixes.
% R = ACOUSTICS.CONVERTUNIT(value, fromUnit, toUnit) converts VALUE from one
% unit to another (same physical dimension). R has fields .value, .from,
% .to, .dimension and .steps.
%
% Metric prefixes are parsed generically onto the base units
% Pa, W, Hz, s, m, g, V, rayl:
% f (1e-15) p (1e-12) n (1e-9) u/μ (1e-6) m (1e-3) c (1e-2) d (1e-1)
% da (1e1) h (1e2) k (1e3) M (1e6) G (1e9) T (1e12)
% e.g. uPa, kPa, MPa, hPa, nm, um, mm, km, mW, uW, pW, fW, kHz, MHz, ms, us.
%
% Also recognised (non-prefixed):
% pressure : bar mbar atm psi mmHg torr dyn/cm2
% length : in inch ft yd mile
% time : min hr h day
% frequency: rad/s rpm
% area : m2 cm2 mm2 km2 volume: m3 L mL cm3 mm3
% speed : m/s km/h ft/s mph knot
% mass : kg t tonne density: kg/m3 g/cm3 g/m3
% intensity: W/m2 mW/m2 uW/m2 nW/m2 pW/m2
% temperature: C K F (affine)
%
% Examples:
% acoustics.convertUnit(2e-5, 'Pa', 'uPa').value % 20 (p_ref)
% acoustics.convertUnit(1e-12, 'W', 'pW').value % 1 (w_ref)
% acoustics.convertUnit(1, 'atm', 'kPa').value % 101.325
% acoustics.convertUnit(1616.3, 'rad/s', 'Hz').value % 257.2
% acoustics.convertUnit(20, 'C', 'K').value % 293.15
%
arguments
value (1,1) double
fromUnit (1,1) string
toUnit (1,1) string
end
a = parseUnit(char(fromUnit));
b = parseUnit(char(toUnit));

if ~isempty(a.temp) || ~isempty(b.temp)
    if isempty(a.temp) || isempty(b.temp)
        error('acoustics:convertUnit:temp', 'Both units must be temperatures (C/K/F).');
    end
    K = toKelvin(value, a.temp);
    R.value = fromKelvin(K, b.temp);
    R.from = char(fromUnit); R.to = char(toUnit); R.dimension = 'temperature';
    R.steps = { sprintf('%g %s = %.6g K = %.6g %s', value, char(fromUnit), ...
        K, R.value, char(toUnit)) };
    return;
end

if ~strcmp(a.dim, b.dim)
    error('acoustics:convertUnit:dim', ...
        'Incompatible units: "%s" is %s but "%s" is %s.', ...
        char(fromUnit), a.dim, char(toUnit), b.dim);
end
base = value * a.factor; % value in the SI base unit
R.value = base / b.factor;
R.from = char(fromUnit); R.to = char(toUnit); R.dimension = a.dim;
R.steps = { ...
    sprintf('%s: 1 %s = %.6g (SI base), 1 %s = %.6g (SI base)', ...
        a.dim, char(fromUnit), a.factor, char(toUnit), b.factor), ...
    sprintf('%g %s = %.6g (SI) = %.6g %s', value, char(fromUnit), base, R.value, char(toUnit))};
end

% =====
function s = parseUnit(u)
```

```

u = strtrim(u);
s.temp = '';
switch u
    case {'C','degC'}, s.temp='C'; s.dim='temperature'; s.factor=1; return;
    case {'K','degK'}, s.temp='K'; s.dim='temperature'; s.factor=1; return;
    case {'F','degF'}, s.temp='F'; s.dim='temperature'; s.factor=1; return;
end
sp = specialMap();
if isKey(sp, u)
    v = sp(u); s.dim = v{1}; s.factor = v{2}; return;
end
bm = baseMap(); pm = prefixMap();
names = keys(bm);
[~, ord] = sort(cellfun(@numel, names), 'descend'); % longest base first
names = names(ord);
for i = 1:numel(names)
    b = names{i};
    if numel(u) >= numel(b) && strcmp(u(end-numel(b)+1:end), b)
        pre = u(1:end-numel(b));
        if isempty(pre)
            pf = 1;
        elseif isKey(pm, pre)
            pf = pm(pre);
        else
            continue; % leading text is not a valid prefix
        end
        v = bm(b); s.dim = v{1}; s.factor = pf*v{2}; return;
    end
end
error('acoustics:convertUnit:unknown', 'Unknown unit: "%s".', u);
end

function m = prefixMap()
    m = containers.Map( ...
        {'f','p','n','u','μ','m','c','d','da','h','k','M','G','T'}, ...
        {1e-15,1e-12,1e-9,1e-6,1e-3,1e-2,1e-1,1e1,1e2,1e3,1e6,1e9,1e12});
end

function m = baseMap()
    m = containers.Map( ...
        {'Pa','W','Hz','s','m','g','V','rayl'}, ...
        {'pressure',1}, {'power',1}, {'frequency',1}, {'time',1}, ...
        {'length',1}, {'mass',1e-3}, {'voltage',1}, {'impedance',1}});
end

function m = specialMap()
    k = {'bar','mbar','atm','psi','mmHg','torr','dyn/cm2', ...
        'in','inch','ft','yd','mile', ...
        'min','hr','h','day', ...
        'rad/s','rpm', ...
        'm2','cm2','mm2','km2','m3','L','mL','cm3','mm3', ...
        'm/s','km/h','ft/s','mph','knot', ...
        'kg','t','tonne','kg/m3','g/cm3','g/m3', ...
        'W/m2','mW/m2','uW/m2','nW/m2','pW/m2'};
    v = [{'pressure',1e5},{'pressure',1e2},{'pressure',101325},{'pressure',6894.757293}, ...
        {'pressure',133.322},{'pressure',133.322},{'pressure',0.1}, ...
        {'length',0.0254},{'length',0.0254},{'length',0.3048},{'length',0.9144},{'length',1609.344}, ...
        {'time',60},{'time',3600},{'time',3600},{'time',86400}, ...
        {'frequency',1/(2*pi)},{'frequency',1/60}, ...
        {'area',1},{'area',1e-4},{'area',1e-6},{'area',1e6}, ...
        {'volume',1},{'volume',1e-3},{'volume',1e-6},{'volume',1e-6},{'volume',1e-9}, ...
        {'speed',1},{'speed',1/3.6},{'speed',0.3048},{'speed',0.44704},{'speed',0.514444}, ...
        {'mass',1},{'mass',1000},{'mass',1000}, ...
        {'density',1},{'density',1000},{'density',1e-3}, ...
        {'intensity',1},{'intensity',1e-3},{'intensity',1e-6},{'intensity',1e-9},{'intensity',1e-12}};
    m = containers.Map(k, v);
end

function K = toKelvin(v, t)
    switch t
        case 'C', K = v + 273.15;
        case 'F', K = (v - 32)*5/9 + 273.15;
        otherwise, K = v; % K
    end
end

function v = fromKelvin(K, t)
    switch t
        case 'C', v = K - 273.15;
        case 'F', v = (K - 273.15)*9/5 + 32;
        otherwise, v = K; % K
    end
end

```

end

Levels: Basic Conversions & Point-Source Relations

Levels: SPL <-> pressure

Convert between sound pressure level and RMS pressure.

```
Lp = 20*log10(p / p_ref) | p = p_ref * 10^(Lp/20)
p_ref = 2e-5 Pa
```

Source file: `+acoustics/splPressure.m`

```
function R = splPressure(opts)
%SPLPRESSURE Convert between sound pressure level and RMS pressure.
% R = ACOUSTICS.SPLPRESSURE('Lp',value) computes RMS pressure from a
% level, and R = ACOUSTICS.SPLPRESSURE('p',value) computes the level
% from an RMS pressure. Supply exactly one.
% Lp = 20*log10(p / p_ref), p = p_ref * 10^(Lp/20), p_ref = 2e-5 Pa
% R has fields .Lp (dB), .p (Pa) and .steps.
%
% Example:
% acoustics.splPressure('p',1).Lp % 93.98 dB (1 Pa)
arguments
opts.Lp double = NaN
opts.p double {mustBePositiveOrNaN(opts.p)} = NaN
end
C = acoustics.constants();
hasLp = ~isnan(opts.Lp);
hasP = ~isnan(opts.p);
if hasLp == hasP
    error('acoustics:splPressure:input', ...
        'Supply exactly one of Lp or p.');
end
if hasP
    R.p = opts.p;
    R.Lp = 20*log10(R.p/C.PREF);
    R.steps = { ...
        'Lp = 20*log10(p / p_ref)', ...
        sprintf('= 20*log10(%4g / 2e-5) = 20*log10(%4g)', R.p, R.p/C.PREF), ...
        sprintf('= %2f dB', R.Lp)};
else
    R.Lp = opts.Lp;
    R.p = C.PREF * 10^(R.Lp/20);
    R.steps = { ...
        'p = p_ref * 10^(Lp/20)', ...
        sprintf('= 2e-5 * 10^(%4g/20) = 2e-5 * %4g', R.Lp, 10^(R.Lp/20)), ...
        sprintf('= %4g Pa', R.p)};
end
end

function mustBePositiveOrNaN(x)
if ~isnan(x) && ~(x > 0)
    error('acoustics:splPressure:pos', 'p must be > 0.');
end
end
```

Levels: Sound power level Lw

Convert between sound power (W) and sound power level (dB).

```
Lw = 10*log10(W / W_ref) | W = W_ref * 10^(Lw/10)
W_ref = 1e-12 W
```

Source file: `+acoustics/powerLevel.m`

```
function R = powerLevel(opts)
%POWERLEVEL Convert between sound power W and sound power level Lw.
% R = ACOUSTICS.POWERLEVEL('W',value) gives Lw; R = ...('Lw',value) gives
% W. Supply exactly one.
% Lw = 10*log10(W / W_ref), W = W_ref * 10^(Lw/10), W_ref = 1e-12 W
% R has fields .W (W), .Lw (dB) and .steps.
%
% Example:
% acoustics.powerLevel('W',0.5).Lw % 116.99 dB
arguments
opts.W double {mustBePositiveOrNaN(opts.W)} = NaN
opts.Lw double = NaN
```

```

end
C = acoustics.constants();
hasW = ~isnan(opts.W);
hasLw = ~isnan(opts.Lw);
if hasW == hasLw
    error('acoustics:powerLevel:input', 'Supply exactly one of W or Lw.');
```

```

end
if hasW
    R.W = opts.W;
    R.Lw = 10*log10(R.W/C.WREF);
    R.steps = { ...
        'Lw = 10*log10(W / W_ref)', ...
        sprintf('= 10*log10(%.4g / 1e-12) = %.2f dB', R.W, R.Lw)};
else
    R.Lw = opts.Lw;
    R.W = C.WREF * 10^(R.Lw/10);
    R.steps = { ...
        'W = W_ref * 10^(Lw/10)', ...
        sprintf('= 1e-12 * 10^(%.4g/10) = %.4g W', R.Lw, R.W)};
end
end
end

function mustBePositiveOrNaN(x)
    if ~isnan(x) && ~(x > 0)
        error('acoustics:powerLevel:pos', 'W must be > 0.');
```

```

    end
end
end

```

Levels: Sound intensity level LI

Sound intensity level, directly from I or from an RMS pressure.

$LI = 10 \cdot \log_{10}(I / I_{\text{ref}})$ | $I_{\text{ref}} = 1e-12 \text{ W/m}^2$

From pressure: | $I = p^2 / (\rho c)$ | [$\rho c = 415 \text{ rayls}$, air, default]

Source file: +acoustics/intensityLevel.m

```

function R = intensityLevel(opts)
%INTENSITYLEVEL Sound intensity level, optionally from RMS pressure.
% R = ACOUSTICS.INTENSITYLEVEL('I',value) uses the intensity directly;
% R = ACOUSTICS.INTENSITYLEVEL('p',value) first forms I = p^2/(rho c)
% with rho c = 415 rayls. Supply exactly one of I or p.
% LI = 10*log10(I / I_ref), I_ref = 1e-12 W/m^2
% R has fields .I (W/m^2), .LI (dB) and .steps.
%
% Example:
% acoustics.intensityLevel('p',1).LI
arguments
    opts.I double {mustBePositiveOrNaN(opts.I)} = NaN
    opts.p double {mustBePositiveOrNaN(opts.p)} = NaN
end
C = acoustics.constants();
hasI = ~isnan(opts.I);
hasP = ~isnan(opts.p);
if hasI == hasP
    error('acoustics:intensityLevel:input', 'Supply exactly one of I or p.');
```

```

end
steps = {};
if hasP
    R.I = opts.p^2 / C.RHOC;
    steps{end+1} = sprintf('I = p_rms^2 / (rho c) = %.4g^2 / %g = %.4g W/m^2', ...
        opts.p, C.RHOC, R.I);
else
    R.I = opts.I;
end
R.LI = 10*log10(R.I/C.IREF);
steps{end+1} = sprintf('LI = 10*log10(I / I_ref) = 10*log10(%.4g / 1e-12) = %.2f dB', ...
    R.I, R.LI);
R.steps = steps;
end

function mustBePositiveOrNaN(x)
    if ~isnan(x) && ~(x > 0)
        error('acoustics:intensityLevel:pos', 'Value must be > 0.');
```

```

    end
end
end

```

Levels: Peak <-> RMS & combine tones

Peak-to-RMS pressure, and quadrature (energy) combination of several tone components.

```
p_rms = P / sqrt(2)
Combine components: | p_tot = sqrt( sum p_i^2 )
```

Source file: `+acoustics/peakToRms.m`

```
function R = peakToRms(P, components)
%PEAKTORMS Peak-to-RMS pressure and quadrature combination of tones.
% R = ACOUSTICS.PEAKTORMS(P) converts a peak pressure amplitude to RMS:
%   p_rms = P / sqrt(2)
% R = ACOUSTICS.PEAKTORMS(P, components) also combines a vector of
% component RMS pressures on a quadrature (energy) basis:
%   p_tot = sqrt( sum p_i^2 )
% R has fields .prms (Pa), .splRms (dB), and when components are given
% .ptot (Pa) and .splTot (dB), plus .steps.
arguments
    P (1,1) double {mustBeNonnegative}
    components (1,:) double = []
end
C = acoustics.constants();
R.prms = P/sqrt(2);
R.splRms = 20*log10(R.prms/C.PREF);
R.steps = { ...
    'p_rms = P / sqrt(2)', ...
    sprintf(' = %g / sqrt(2) = %.4g Pa -> SPL = %.2f dB', P, R.prms, R.splRms)};
if ~isempty(components)
    sumSq = sum(components.^2);
    R.ptot = sqrt(sumSq);
    R.splTot = 20*log10(R.ptot/C.PREF);
    terms = strjoin(arrayfun(@(x) sprintf('%.4g^2', x), components, ...
        'UniformOutput', false), ' + ');
    R.steps = [R.steps, { ...
        'p_tot = sqrt( sum p_i^2 )', ...
        sprintf(' = sqrt( %s ) = sqrt(%.4g) = %.4g Pa', terms, sumSq, R.ptot), ...
        sprintf('SPL_tot = 20*log10(%.4g / 2e-5) = %.2f dB', R.ptot, R.splTot)}];
end
end
```

Levels: PSD -> RMS pressure

RMS pressure and SPL from a straight-line power-spectral-density segment.

```
p_rms^2 = 1/2 (S1 + S2)(f2 - f1) | [trapezoidal rule, S in Pa^2/Hz]
SPL = 20*log10(p_rms / p_ref)
```

Source file: `+acoustics/psdToRms.m`

```
function R = psdToRms(f1, f2, S1, S2)
%PSDTORMS RMS pressure and SPL from a power spectral density band.
% R = ACOUSTICS.PSDTORMS(f1, f2, S1, S2) integrates a straight-line PSD
% segment (Pa^2/Hz) between f1 and f2 (Hz) using the trapezoidal rule:
%   p_rms^2 = 1/2 (S1 + S2)(f2 - f1)
%   SPL = 20*log10(p_rms / p_ref)
% R has fields .meanSquare (Pa^2), .prms (Pa), .spl (dB) and .steps.
arguments
    f1 (1,1) double {mustBeNonnegative}
    f2 (1,1) double {mustBeNonnegative}
    S1 (1,1) double {mustBeNonnegative}
    S2 (1,1) double {mustBeNonnegative}
end
if ~(f2 > f1)
    error('acoustics:psdToRms:band', 'Upper frequency must exceed lower frequency.');
```

```
end
C = acoustics.constants();
bw = f2 - f1;
R.meanSquare = (S1 + S2)/2 * bw;
R.prms = sqrt(R.meanSquare);
R.spl = 20*log10(R.prms/C.PREF);
R.steps = { ...
    sprintf('p_rms^2 = 1/2 (S1+S2)(f2-f1) = 1/2 (%.4g + %.4g)(%.4g)', S1, S2, bw), ...
    sprintf(' = %.4g Pa^2', R.meanSquare), ...
    sprintf('p_rms = sqrt(%.4g) = %.4g Pa', R.meanSquare, R.prms), ...
    sprintf('SPL = 20*log10(%.4g / 2e-5) = %.2f dB', R.prms, R.spl)};
end
```

Levels: Radiated power (point source)

Point-source radiation relations at a distance: give ONE of W, I or peak pressure P, get the rest.

```
S = 4*pi*r^2 / Q | (radiating area; Q: 1 free field, 2 hemisphere, 4 edge, 8 corner)
From P: | p_rms = P/sqrt(2), | I = p_rms^2/(rho c) | From W: | I = W / S
```


$p_{rms} = \sqrt{I \cdot \rho c}$ | $W = I \cdot S$
 $SPL = 20 \cdot \log_{10}(p_{rms}/2e-5)$ | $SIL = 10 \cdot \log_{10}(I/1e-12)$ | $Lw = 10 \cdot \log_{10}(W/1e-12)$

Use case: 'A spherical source radiates W watts... find p_{rms} , SPL, I, SIL at distance r.'

Source file: `acoustics/radiatedPower.m`

```
function R = radiatedPower(r, opts)
%RADIATEDPOWER Point-source radiation relations at a distance (both ways).
% Give exactly ONE of the source quantities and get the full set at
% distance r (m) for directivity Q (1 free field / 2 hemisphere / 4 edge /
% 8 corner):
% R = ACOUSTICS.RADIATEDPOWER(r, 'W', value) % sound POWER (W) -> p,SPL,I,SIL
% R = ACOUSTICS.RADIATEDPOWER(r, 'I', value) % intensity (W/m^2)
% R = ACOUSTICS.RADIATEDPOWER(r, 'P', value) % PEAK pressure (Pa)
%
% Relations (rho c = 415 rayls by default, 'rhoc' overrides):
% S = 4*pi*r^2 / Q (radiating area)
% from P: p_rms = P/sqrt(2), I = p_rms^2/(rho c)
% from W: I = W / S
% p_rms = sqrt(I*rho c), W = I*S
% SPL = 20*log10(p_rms/2e-5), SIL = 10*log10(I/1e-12), Lw = 10*log10(W/1e-12)
%
% R has fields .I, .prms, .spl, .sil, .area, .W, .Lw and .steps.
%
% Example (W = 1.04 W, r = 0.17 m, Q = 1):
% R = acoustics.radiatedPower(0.17,'W',1.04);
% R.prms % 34.47 Pa R.spl % 124.73 dB
% R.I % 2.864 W/m^2 R.sil % 124.57 dB
% Example (P = 25 Pa, r = 2 m, Q = 1):
% R = acoustics.radiatedPower(2,'P',25); R.I % 0.753 R.W % 37.85
arguments
r (1,1) double {mustBePositive}
opts.W double = NaN
opts.I double = NaN
opts.P double = NaN
opts.Q (1,1) double {mustBePositive} = 1
opts.rhoc (1,1) double {mustBePositive} = 415
end
C = acoustics.constants();
given = [~isnan(opts.W), ~isnan(opts.I), ~isnan(opts.P)];
if sum(given) ~= 1
error('acoustics:radiatedPower:input', 'Supply exactly one of W, I or P.');
```

```
end
R.area = 4*pi*r^2/opts.Q;
steps = { sprintf('S = 4*pi*r^2/Q = 4*pi*g^2/%g = %.4g m^2', r, opts.Q, R.area) };
if ~isnan(opts.P)
prms = opts.P/sqrt(2);
R.I = prms^2/opts.rhoc;
steps = [{ ...
sprintf('p_rms = P/sqrt(2) = %g/sqrt(2) = %.4g Pa', opts.P, prms), ...
sprintf('I = p_rms^2/(rho c) = %.4g^2/%g = %.4g W/m^2', prms, opts.rhoc, R.I)}, steps];
elseif ~isnan(opts.W)
R.I = opts.W/R.area;
steps[end+1] = sprintf('I = W/S = %g/%.4g = %.4g W/m^2', opts.W, R.area, R.I);
else
R.I = opts.I;
end
R.prms = sqrt(R.I*opts.rhoc);
R.W = R.I*R.area;
R.spl = 20*log10(R.prms/C.PREF);
R.sil = 10*log10(R.I/C.IREF);
R.Lw = 10*log10(R.W/C.WREF);
R.steps = [steps, { ...
sprintf('p_rms = sqrt(I*rho c) = sqrt(%g*%g) = %.4g Pa', R.I, opts.rhoc, R.prms), ...
sprintf('SPL = 20*log10(p_rms/2e-5) = %.2f dB', R.spl), ...
sprintf('I = %.4g W/m^2 SIL = 10*log10(I/1e-12) = %.2f dB', R.I, R.sil), ...
sprintf('W = I*S = %.4g W Lw = 10*log10(W/1e-12) = %.2f dB', R.W, R.Lw)}];
end
```

Levels: Power into a medium (pipe/interface)

Sound power carried into a second medium (e.g. water) through an anechoically-terminated pipe.

$p_{rms} = p_{ref} \cdot 10^{(SPL/20)}$ | [continuous across the interface]
 $I = p_{rms}^2 / (\rho c)_{medium}$ | $W = I \cdot A$, | $A = \pi \cdot d^2/4$
 $Lw = 10 \cdot \log_{10}(W / 1e-12)$

Use case: pipe immersed in a lake, perforated to act anechoically; SPL measured at the interface.

Source file: `acoustics/powerIntoMedium.m`

```
function R = powerIntoMedium(SPL, d, opts)
%POWERINTOMEDIUM Sound power carried into a medium through a pipe (from SPL).
% R = ACOUSTICS.POWERINTOMEDIUM(SPL, d) for a pipe of internal diameter d (m)
% whose end is coupled to a medium (default water) that is anechoically
% terminated (a travelling wave, no reflections). A microphone at the
% interface measures an RMS sound pressure level SPL (dB re 2e-5 Pa); the
% acoustic pressure is continuous across the interface, so:
%   p_rms = p_ref * 10^(SPL/20)
%   I      = p_rms^2 / (rho * c)      [travelling wave in the medium]
%   W      = I * A,   A = pi*d^2/4   [pipe cross-section]
%   Lw     = 10*log10(W / 1e-12)
% Defaults are for water: 'rho' 1000 kg/m^3, 'c' 1480 m/s ('pref' 2e-5 Pa).
%
% R has fields .prms (Pa), .rhoc, .area (m^2), .I (W/m^2), .W (W), .Lw (dB)
% and .steps.
%
% Example (80 mm pipe into a lake, 153 dB at the surface):
%   acoustics.powerIntoMedium(153, 0.080).W   % 2.71e-3 W
arguments
SPL (1,1) double
d (1,1) double {mustBePositive}
opts.rho (1,1) double {mustBePositive} = 1000
opts.c (1,1) double {mustBePositive} = 1480
opts.pref (1,1) double {mustBePositive} = 2e-5
end
C = acoustics.constants();
R.prms = opts.pref * 10^(SPL/20);
R.rhoc = opts.rho*opts.c;
R.area = pi*d^2/4;
R.I = R.prms^2 / R.rhoc;
R.W = R.I * R.area;
R.Lw = 10*log10(R.W/C.WREF);
R.steps = { ...
'Anechoic medium (travelling wave), pressure continuous at interface', ...
sprintf('p_rms = p_ref*10^(SPL/20) = 2e-5*10^(%.4g/20) = %.1f Pa', SPL, R.prms), ...
sprintf('rho c = %.4g rayl . A = pi*d^2/4 = %.5g m^2', R.rhoc, R.area), ...
sprintf('I = p_rms^2/(rho c) = %.4g W/m^2', R.I), ...
sprintf('W = I*A = %.4g W (%.3f mW)', R.W, R.W*1000), ...
sprintf('Lw = 10*log10(W/1e-12) = %.1f dB', R.Lw)};
end
```

Combine & Subtract Sources

Combine: add sound levels

Energy (incoherent) sum of any list of decibel levels.

```
L_tot = 10*log10( sum 10^(Li/10) )
```

Source file: `acoustics/combineLevels.m`

```
function R = combineLevels(levels)
%COMBINELEVELS Energy (incoherent) sum of decibel levels.
% R = ACOUSTICS.COMBINELEVELS(levels) combines a vector of levels (dB)
% on an energy basis: L_tot = 10*log10( sum 10^(Li/10) ).
% R has fields:
%   .total    combined level (dB)
%   .pressure equivalent RMS pressure (Pa), using p_ref = 2e-5
%   .steps    worked-step strings
%
% Example:
%   acoustics.combineLevels([80 80 74]).total   % 83.44 dB
arguments
levels (1,:) double
end
C = acoustics.constants();
e = 10.^(levels/10);
s = sum(e);
R.total = 10*log10(s);
R.pressure = C.PREF * 10^(R.total/20);
terms = strjoin(arrayfun(@(x) sprintf('10^(%.4g/10)', x), levels, ...
'UniformOutput', false), ' + ');
R.steps = { ...
'L_tot = 10*log10( sum 10^(Li/10) )', ...
sprintf('= 10*log10( %s )', terms), ...
sprintf('= 10*log10( %.5g ) = %.2f dB', s, R.total)};
end
```

Combine: N identical sources

Combined level of N identical incoherent sources, from one source's level.

$$L_{\text{tot}} = L_1 + 10 \cdot \log_{10}(N)$$

Source file: `acoustics/nIdenticalSources.m`

```
function R = nIdenticalSources(L1, N)
%NIDENTICALSOURCES Combined level of N identical incoherent sources.
% R = ACOUSTICS.NIDENTICALSOURCES(L1, N) returns
% L_tot = L1 + 10*log10(N)
% R has fields .total (dB) and .steps.
arguments
    L1 (1,1) double
    N (1,1) double {mustBePositive}
end
R.total = L1 + 10*log10(N);
R.steps = { ...
    'L_tot = L1 + 10*log10(N)', ...
    sprintf(' = %.4g + 10*log10(%g) = %.4g + %.4g = %.2f dB', ...
        L1, N, L1, 10*log10(N), R.total)};
end
```

Combine: increase from more sources

Level rise when 'added' extra identical sources join N_1 existing ones.

$$N_2 = N_1 + \text{added}$$
$$dL = 10 \cdot \log_{10}(N_2/N_1) \quad | \quad L_{\text{new}} = L_1 + dL$$

Source file: `acoustics/increaseFromSources.m`

```
function R = increaseFromSources(N1, L1, added)
%INCREASEFROMSOURCES Level rise when more identical sources are added.
% R = ACOUSTICS.INCREASEFROMSOURCES(N1, L1, added) returns the change and
% new level when 'added' extra sources join  $N_1$  existing ones measured at
% level  $L_1$ :
% N2 = N1 + added
% dL = 10*log10(N2/N1), L_new = L1 + dL
% R has fields .N2, .delta (dB), .newLevel (dB) and .steps.
arguments
    N1 (1,1) double {mustBePositive}
    L1 (1,1) double
    added (1,1) double
end
R.N2 = N1 + added;
if ~(R.N2 > 0)
    error('acoustics:increaseFromSources:count', 'Resulting count must be positive.');
R.delta = 10*log10(R.N2/N1);
R.newLevel = L1 + R.delta;
R.steps = { ...
    sprintf('N2 = N1 + added = %g + %g = %g', N1, added, R.N2), ...
    sprintf('dL = 10*log10(N2/N1) = 10*log10(%g) = %.3f dB', R.N2/N1, R.delta), ...
    sprintf('L_new = L1 + dL = %.4g + %.3f = %.3f dB', L1, R.delta, R.newLevel)};
end
```

Combine: error using larger signal only

Fractional error made by ignoring the smaller of two signals (ratio = $p_2/p_1 \leq 1$).

$$p_{\text{tot}} = p_1 \cdot \sqrt{1 + \text{ratio}^2}$$
$$\text{error} = 1/\sqrt{1 + \text{ratio}^2} - 1$$

Source file: `acoustics/largerSignalError.m`

```
function R = largerSignalError(ratio)
%LARGERSIGNALERROR Error from ignoring the smaller of two signals.
% R = ACOUSTICS.LARGERSIGNALERROR(ratio) takes ratio =  $p_2/p_1$  (smaller /
% larger,  $0 \leq \text{ratio} \leq 1$ ) and returns the fractional under-estimate made
% by keeping only the larger signal:
% p_tot = p1 * sqrt(1 + ratio^2)
% error = 1/sqrt(1 + ratio^2) - 1
% R has fields .ptotFactor (x p1), .errorPct (%) and .steps.
arguments
    ratio (1,1) double {mustBeNonnegative}
end
if ratio > 1
    error('acoustics:largerSignalError:ratio', 'ratio (p2/p1) must be <= 1.');
R.ptotFactor = sqrt(1 + ratio^2);
```

```

R.errorPct = (1/R.ptotFactor - 1)*100;
R.steps = { ...
    sprintf('r = p2/p1 = %.4g', ratio), ...
    sprintf('p_tot = p1*sqrt(1 + r^2) = p1*sqrt(1 + %.4g) = %.5g*p1', ratio^2, R.ptotFactor), ...
    'Error = 1/sqrt(1 + r^2) - 1', ...
    sprintf('= 1/%.5g - 1 = %.2f %%', R.ptotFactor, R.errorPct)};
end

```

Combine: max sources under a limit

Largest number of identical sources whose combined level stays under a permitted limit.

```

one source: L1 = Ltot - 10*log10(N1)
N <= N1 * 10^((Lmax - Ltot)/10) | [round down]

```

Source file: `+acoustics/maxSourcesUnderLimit.m`

```

function R = maxSourcesUnderLimit(N1, Ltot, Lmax)
%MAXSOURCESUNDERLIMIT Largest number of identical sources under a limit.
% R = ACOUSTICS.MAXSOURCESUNDERLIMIT(N1, Ltot, Lmax) where N1 identical
% sources currently produce Ltot (dB) and Lmax is the permitted level:
% one source: L1 = Ltot - 10*log10(N1)
% N <= N1 * 10^((Lmax - Ltot)/10) (rounded down)
% R has fields .perSource (dB), .Nexact, .N (integer), .levelAtN (dB),
% .levelAtNplus1 (dB) and .steps.
arguments
    N1 (1,1) double {mustBePositive}
    Ltot (1,1) double
    Lmax (1,1) double
end
R.perSource = Ltot - 10*log10(N1);
R.Nexact = N1 * 10^((Lmax - Ltot)/10);
R.N = floor(R.Nexact + 1e-9);
if R.N < 1
    R.levelAtN = NaN; R.levelAtNplus1 = NaN;
    R.steps = {sprintf('Even one source (%.2f dB) exceeds the %.4g dB limit.', ...
        R.perSource, Lmax)};
    return;
end
R.levelAtN = Ltot + 10*log10(R.N/N1);
R.levelAtNplus1 = Ltot + 10*log10((R.N+1)/N1);
R.steps = { ...
    sprintf('One source: L1 = Ltot - 10*log10(N1) = %.4g - 10*log10(%g) = %.2f dB', ...
        Ltot, N1, R.perSource), ...
    'N <= N1 * 10^((Lmax - Ltot)/10)', ...
    sprintf('= %g * 10^((%.4g - %.4g)/10) = %.3f -> round down = %d', ...
        N1, Lmax, Ltot, R.Nexact, R.N), ...
    sprintf('Level at %d = %.2f dB (<= %.4g) · at %d = %.2f dB (over)', ...
        R.N, R.levelAtN, Lmax, R.N+1, R.levelAtNplus1)};
end

```

Subtract: remove background / source

Energy-subtract one level (e.g. background noise) from a measured total.

```

L_remaining = 10*log10( 10^(Ltot/10) - 10^(Lremove/10) )

```

Source file: `+acoustics/subtractLevels.m`

```

function R = subtractLevels(Ltot, Lremove)
%SUBTRACTLEVELS Energy subtraction of one level from a total.
% R = ACOUSTICS.SUBTRACTLEVELS(Ltot, Lremove) removes the contribution of
% Lremove (e.g. background) from a measured total Ltot:
% L_rem = 10*log10( 10^(Ltot/10) - 10^(Lremove/10) )
% R has fields .remaining (dB) and .steps.
arguments
    Ltot (1,1) double
    Lremove (1,1) double
end
diff = 10^(Ltot/10) - 10^(Lremove/10);
if diff <= 0
    error('acoustics:subtractLevels:order', ...
        'Total must exceed the level being removed.');
```

```

end
R.remaining = 10*log10(diff);
R.steps = { ...
    'L_rem = 10*log10( 10^(Ltot/10) - 10^(Lremove/10) )', ...
    sprintf('= 10*log10( %.4g - %.4g ) = 10*log10( %.4g )', ...
        10^(Ltot/10), 10^(Lremove/10), diff), ...
    sprintf('= %.2f dB', R.remaining)};
end

```

Subtract: one of N identical sources

Level of a single source, given the combined level of N identical ones.

$$L_1 = L_{\text{tot}} - 10 \cdot \log_{10}(N)$$

Use case: 'two identical fans give 82 dB together; find the level of one fan.'

Source file: `+acoustics/oneOfNSources.m`

```
function R = oneOfNSources(Ltot, N)
%ONEOFNSOURCES Level of a single source among N identical ones.
% R = ACOUSTICS.ONEOFNSOURCES(Ltot, N) returns
%   L1 = Ltot - 10*log10(N)
% R has fields .each (dB) and .steps.
arguments
    Ltot (1,1) double
    N     (1,1) double {mustBePositive}
end
R.each = Ltot - 10*log10(N);
R.steps = { ...
    'L1 = Ltot - 10*log10(N)', ...
    sprintf(' = %.4g - 10*log10(%g) = %.4g - %.4g = %.2f dB', ...
        Ltot, N, Ltot, 10*log10(N), R.each)};
end
```

Waves

Waves: $c = f \times \lambda$

Solve the basic wave relation for whichever quantity is missing; also reports T , ω , k .

$$c = f \cdot \lambda \mid T = 1/f \mid \omega = 2\pi \cdot f \mid k = \omega/c = 2\pi/\lambda$$

Source file: `+acoustics/waveRelation.m`

```
function R = waveRelation(opts)
%WAVERELATION Solve c = f*lambda for whichever term is missing.
% R = ACOUSTICS.WAVERELATION('c',...,'f',...) etc. Supply exactly two of
% c (m/s), f (Hz), lambda (m); the third is solved from c = f*lambda.
% R also reports period T, angular frequency omega and wavenumber k.
% R has fields .c, .f, .lambda, .T, .omega, .k and .steps.
%
% Example:
% acoustics.waveRelation('c',343,'f',1000).lambda % 0.343 m
arguments
    opts.c      double = NaN
    opts.f      double = NaN
    opts.lambda double = NaN
end
c = opts.c; f = opts.f; lam = opts.lambda;
known = ~isnan(c) + ~isnan(f) + ~isnan(lam);
if known < 2
    error('acoustics:waveRelation:input', ...
        'Supply at least two of c, f, lambda.');
```

```
end
if isnan(c), c = f*lam;
elseif isnan(f), f = c/lam;
elseif isnan(lam), lam = c/f;
end
R.c = c; R.f = f; R.lambda = lam;
R.T = 1/f; R.omega = 2*pi*f; R.k = 2*pi/lam;
R.steps = { ...
    sprintf('c = f*lambda -> c = %.3f m/s, f = %.3f Hz, lambda = %.4f m', c, f, lam), ...
    sprintf('T = 1/f = %.4g s', R.T), ...
    sprintf('omega = 2*pi*f = %.1f rad/s', R.omega), ...
    sprintf('k = 2*pi/lambda = %.3f rad/m', R.k)};
end
```

Waves: speed of sound from temperature

Speed of sound from air temperature, or (inverse) temperature from a measured speed / travel time.

$$\text{Forward: } T_0 = T_c + 273.2 \text{ (K)}, \mid c = \sqrt{\gamma \cdot R \cdot T_0} \mid [\gamma=1.4, R=287 \text{ J/kg/K}]$$

$$\text{Inverse (from measured c, or distance/time } c=d/t): \mid T_0 = c^2/(\gamma \cdot R), \mid T_c = T_0 - 273.2$$

Use case: an impulse crosses d metres between two probe microphones in time t -> estimate air temperature.

Source file: `+acoustics/speedOfSoundTemp.m`

```

function R = speedOfSoundTemp(Tc, opts)
%SPEEDOFSOUNDTEMP Speed of sound in a gas and temperature (either direction).
% FORWARD - speed from temperature:
%   R = ACOUSTICS.SPEEDOFSOUNDTEMP(Tc)           % Tc in degrees Celsius
%   T0 = Tc + 273.2 (K), c = sqrt(gamma*R*T0)
%
% INVERSE - temperature from speed:
%   R = ACOUSTICS.SPEEDOFSOUNDTEMP('c', c)       % measured speed (m/s)
%   R = ACOUSTICS.SPEEDOFSOUNDTEMP('distance',d,'time',t) % c = d/t
%   T0 = c^2/(gamma*R), Tc = T0 - 273.2
% e.g. an impulse crossing d = 8 m between two probe microphones in t = 20 ms
% gives c = 400 m/s and Tc ~ 125 C.
%
% Optional 'gamma' (1.4) and 'R' (287 J/kg/K) override the gas properties.
% R has fields .Tc (C), .T0 (K), .c (m/s), .cShortcut (20.06*sqrt(T0)) and
% .steps.
%
% Examples:
%   acoustics.speedOfSoundTemp(20).c             % 343.2 m/s
%   acoustics.speedOfSoundTemp('distance',8,'time',0.02).Tc % ~125 C
arguments
Tc double = NaN
opts.c      (1,1) double = NaN
opts.distance (1,1) double = NaN
opts.time    (1,1) double = NaN
opts.gamma   (1,1) double {mustBePositive} = 1.4
opts.R       (1,1) double {mustBePositive} = 287
end
g = opts.gamma; Rgas = opts.R;

% Work out the measured speed, if any was supplied.
c = opts.c;
dtStep = {};
if isnan(c) && ~isnan(opts.distance) && ~isnan(opts.time)
    if opts.time <= 0
        error('acoustics:speedOfSoundTemp:time', 'time must be > 0.');
    end
    c = opts.distance/opts.time;
    dtStep = { sprintf('c = distance/time = %g/%g = %.2f m/s', ...
        opts.distance, opts.time, c) };
end

if ~isnan(Tc)
    % ---- forward: temperature -> speed ----
    R.Tc = Tc;
    R.T0 = Tc + 273.2;
    R.c = sqrt(g*Rgas*R.T0);
    R.cShortcut = 20.06*sqrt(R.T0);
    R.steps = { ...
        sprintf('T0 = Tc + 273.2 = %.1f K', R.T0), ...
        sprintf('c = sqrt(gamma*R*T0) = sqrt(%g*%g*.1f) = %.2f m/s', ...
            g, Rgas, R.T0, R.c), ...
        sprintf('(air shortcut 20.06*sqrt(T0) = %.2f m/s)', R.cShortcut)};
elseif ~isnan(c)
    % ---- inverse: speed -> temperature ----
    R.c = c;
    R.T0 = c^2/(g*Rgas);
    R.Tc = R.T0 - 273.2;
    R.cShortcut = 20.06*sqrt(R.T0);
    R.steps = [ dtStep, { ...
        sprintf('T0 = c^2/(gamma*R) = %.2f^2/(%g*%g) = %.2f K', c, g, Rgas, R.T0), ...
        sprintf('Tc = T0 - 273.2 = %.1f degC', R.Tc) } ];
else
    error('acoustics:speedOfSoundTemp:input', ...
        'Give Tc (temperature), or c, or distance and time.');
end
end

```

Waves: particle velocity & displacement

Particle velocity, displacement, intensity and SPL of a plane wave from its peak pressure.

```

u = P / (rho * c) | particle velocity amplitude
xi = u / omega, | omega = 2*pi*f | displacement amplitude
I = P^2 / (2 * rho * c) | intensity
p_rms = P / sqrt(2) | Lp = 20*log10(p_rms / p_ref)

```

Source file: `acoustics/particleMotion.m`

```

function R = particleMotion(P, f, opts)
%PARTICLEMOTION Particle velocity, displacement and intensity of a wave.

```

```
% R = ACOUSTICS.PARTICLEMOTION(P, f) for a plane wave of PEAK pressure
% amplitude P (Pa) at frequency f (Hz), with rho c = 415 rayls by default:
% u = P / (rho c) particle velocity amplitude
% xi = u / omega, omega = 2*pi*f displacement amplitude
% I = P^2 / (2 rho c) intensity
% prms = P / sqrt(2) RMS pressure
% Lp = 20*log10(prms / p_ref) sound pressure level (p_ref = 2e-5)
% Optional 'rhoc' overrides the impedance. R has fields .u, .xi, .I,
% .prms, .Lp, .omega and .steps.
arguments
    P (1,1) double {mustBeNonnegative}
    f (1,1) double {mustBePositive}
    opts.rhoc (1,1) double {mustBePositive} = 415
end
C = acoustics.constants();
R.omega = 2*pi*f;
R.u = P/opts.rhoc;
R.xi = R.u/R.omega;
R.I = P^2/(2*opts.rhoc);
R.prms = P/sqrt(2);
R.Lp = 20*log10(R.prms/C.PREF);
R.steps = { ...
    sprintf('(velocity) u = P/(rho c) = %g/%g = %.4g m/s', P, opts.rhoc, R.u), ...
    sprintf('(displacement) xi = u/omega = u/(2*pi*f) = %.4g m', R.xi), ...
    sprintf('(intensity) I = P^2/(2 rho c) = %.4g W/m^2', R.I), ...
    sprintf('(RMS pressure) p_rms = P/sqrt(2) = %.4g Pa', R.prms), ...
    sprintf('(SPL) Lp = 20*log10(p_rms/2e-5) = %.2f dB', R.Lp)};
end
```

Waves: plane wave u -> pressure -> SPL (air/water)

RMS particle velocity of a plane progressive wave -> pressure -> SPL. in air OR water.

```
p = rho * c * u | Lp = 20*log10(p / p_ref)
Air: | rho=1.21, c=343, | p_ref=2e-5 Pa (20 uPa)
Water: rho=1000, c=1500, p_ref=1e-6 Pa (1 uPa)
```

Source file: `acoustics/planeWaveLevel.m`

```
function R = planeWaveLevel(u, opts)
%PLANEWAVELEVEL Plane-wave particle velocity -> pressure -> sound pressure level.
% R = ACOUSTICS.PLANEWAVELEVEL(u) takes the RMS particle velocity u (m/s) of a
% plane progressive wave and returns the RMS pressure and the sound pressure
% level, using p = rho*c*u and Lp = 20*log10(p/p_ref).
%
% Options:
% 'rho' medium density (kg/m^3), default 1.21 (air)
% 'c' speed of sound (m/s), default 343 (air)
% 'pref' reference pressure (Pa), default 2e-5 (20 uPa, air standard)
% For WATER use rho=1000, c=1500 and pref=1e-6 (1 uPa, the underwater
% acoustics reference).
%
% R has fields .p (Pa), .Lp (dB), .rhoc (rayls) and .steps.
%
% Examples (u = 0.11 m/s):
% acoustics.planeWaveLevel(0.11).Lp % 127.17 dB (air)
% acoustics.planeWaveLevel(0.11,'rho',1000,'c',1500,'pref',1e-6).Lp % 224.35 dB (water)
arguments
    u (1,1) double {mustBeNonnegative}
    opts.rho (1,1) double {mustBePositive} = 1.21
    opts.c (1,1) double {mustBePositive} = 343
    opts.pref (1,1) double {mustBePositive} = 2e-5
end
R.rhoc = opts.rho * opts.c;
R.p = R.rhoc * u;
R.Lp = 20*log10(R.p / opts.pref);
R.steps = { ...
    sprintf('rho c = %g * %g = %.5g rayls', opts.rho, opts.c, R.rhoc), ...
    sprintf('p = rho c * u = %.5g * %g = %.5g Pa', R.rhoc, u, R.p), ...
    sprintf('Lp = 20*log10(p / p_ref) = 20*log10(%.5g / %g)', R.p, opts.pref), ...
    sprintf(' = %.2f dB', R.Lp)};
end
```

Waves: octave band edges & pipe modes

Lower/upper edges and bandwidth of an octave or 1/3-octave band about a centre frequency.

```
Octave: | f_lower = fc/sqrt(2), | f_upper = fc*sqrt(2) | -> %BW = 70.7% (constant)
1/3-octave: | ratio 2^(1/6) | -> %BW = 23.1% (constant)
```

Source file: +acoustics/octaveBandEdges.m

```
function R = octaveBandEdges(fc, opts)
%OCTAVEBANDEDGES Lower/upper edges and bandwidth of an octave-type band.
% R = ACOUSTICS.OCTAVEBANDEDGES(fc) for an octave band centred at fc (Hz):
%   f_lower = fc/sqrt(2), f_upper = fc*sqrt(2)
% R = ACOUSTICS.OCTAVEBANDEDGES(fc,'type','third') uses a 1/3-octave band
% with ratio 2^(1/6). R has fields .lower, .upper (Hz), .bandwidth (Hz),
% .percent (% of fc) and .steps.
arguments
    fc (1,1) double {mustBePositive}
    opts.type (1,1) string {mustBeMember(opts.type,["octave","third"])} = "octave"
end
if opts.type == "third"
    k = 2^(1/6); ks = '2^(1/6)'; nm = 'one-third octave'; ref = '23.1';
else
    k = sqrt(2); ks = 'sqrt(2)'; nm = 'octave'; ref = '70.7';
end
R.lower = fc/k;
R.upper = fc*k;
R.bandwidth = R.upper - R.lower;
R.percent = R.bandwidth/fc*100;
R.steps = { ...
    sprintf('f_lower = fc/%s = %.1f Hz', ks, R.lower), ...
    sprintf('f_upper = fc*s = %.1f Hz', ks, R.upper), ...
    sprintf('BW = %.1f - %.1f = %.1f Hz', R.upper, R.lower, R.bandwidth), ...
    sprintf('%sBW = BW/fc*100 = %.1f %% (constant -> %s ~ %s %%)', R.percent, nm, ref)};
end
```

Waves: pipe natural frequencies (ends)

Natural (resonant) frequencies of an air column in a pipe, by end condition.

closed-open | (quarter-wave): | $f_n = (2n-1) c / (4L)$, | $n = 1, 2, 3, \dots$

open-open | (half-wave): | $f_n = n c / (2L)$

closed-closed (half-wave): | $f_n = n c / (2L)$

Pressure node: centre ($L/2$) for closed-closed; the open end(s) otherwise

Source file: +acoustics/pipeModes.m

```
function R = pipeModes(L, opts)
%PIPEMODES Natural (resonant) frequencies of an air column in a pipe.
% R = ACOUSTICS.PIPEMODES(L) returns the first natural frequencies of a pipe
% of length L (m). The end conditions set the formula ('ends' option):
%   "closed-open" (quarter-wave) f_n = (2n-1) c / (4L) n = 1,2,3...
%   "open-open" (half-wave) f_n = n c / (2L)
%   "closed-closed" (half-wave) f_n = n c / (2L)
% The default is "closed-open". Optional 'c' (343 m/s) and 'n' (3 modes).
%
% R has fields:
%   .f natural frequencies (Hz, 1xn)
%   .omega angular frequencies (rad/s, 1xn)
%   .nodesFundamental x-position(s) of the pressure node(s) of the
%   fundamental mode (m): centre L/2 for closed-closed,
%   the open end(s) otherwise
%   .steps
%
% Example (2 m pipe closed at both ends):
% R = acoustics.pipeModes(2, 'ends', "closed-closed");
% R.omega % [538.8 1077.6 1616.3] rad/s
% R.nodesFundamental % 1 (pressure node at the centre)
arguments
    L (1,1) double {mustBePositive}
    opts.c (1,1) double {mustBePositive} = 343
    opts.n (1,1) double {mustBePositive, mustBeInteger} = 3
    opts.ends (1,1) string ...
        {mustBeMember(opts.ends,["closed-open","open-open","closed-closed"])} = "closed-open"
end
n = 1:opts.n;
switch opts.ends
case "closed-open"
    R.f = (2*n - 1)*opts.c/(4*L);
    rel = 'f_n = (2n-1) c / (4L)';
    R.nodesFundamental = L; % pressure node at the open end
otherwise
    % open-open / closed-closed
    R.f = n*opts.c/(2*L);
    rel = 'f_n = n c / (2L)';
    if opts.ends == "closed-closed"
        R.nodesFundamental = L/2; % pressure node at the centre
    else
        R.nodesFundamental = [0 L]; % pressure nodes at the open ends
    end
end
```



```

end
R.omega = 2*pi*R.f;

lines = {sprintf('%s pipe: %s (omega = 2*pi*f)', opts.ends, rel)};
for k = n
    lines{end+1} = sprintf(' f%d = %.2f Hz omega%d = %.1f rad/s', ...
        k, R.f(k), k, R.omega(k)); %#ok<AGROW>
end
lines{end+1} = sprintf('Fundamental pressure node(s) at x = %s m', ...
    strjoin(arrayfun(@(x) sprintf('%.3g', x), R.nodesFundamental, ...
        'UniformOutput', false), ', '));
R.steps = lines;
end

```

Distance Propagation

Distance: attenuation L2 at new distance

Propagate a measured level out to a new distance, for a point or a line source.

Point (spherical, -6 dB/doubling): $L_2 = L_1 - 20 \cdot \log_{10}(r_2/r_1)$
 Line | (cylindrical, -3 dB/doubling): $L_2 = L_1 - 10 \cdot \log_{10}(r_2/r_1)$

Source file: `acoustics/distanceAttenuation.m`

```

function R = distanceAttenuation(L1, r1, r2)
%DISTANCEATTENUATION Level at a new distance for point and line sources.
% R = ACOUSTICS.DISTANCEATTENUATION(L1, r1, r2) propagates a level L1 (dB)
% measured at r1 (m) out to r2 (m):
% point (spherical, -6 dB/doubling): L2 = L1 - 20*log10(r2/r1)
% line (cylindrical, -3 dB/doubling): L2 = L1 - 10*log10(r2/r1)
% R has fields .point (dB), .line (dB) and .steps.
arguments
    L1 (1,1) double
    r1 (1,1) double {mustBePositive}
    r2 (1,1) double {mustBePositive}
end
ratio = log10(r2/r1);
R.point = L1 - 20*ratio;
R.line = L1 - 10*ratio;
R.steps = { ...
    sprintf('log10(r2/r1) = log10(%g/%g) = %.4f', r2, r1, ratio), ...
    sprintf('Point: L2 = L1 - 20*log10(r2/r1) = %.4g - %.2f = %.2f dB', ...
        L1, 20*ratio, R.point), ...
    sprintf('Line: L2 = L1 - 10*log10(r2/r1) = %.4g - %.2f = %.2f dB', ...
        L1, 10*ratio, R.line)};
end

```

Distance: solve distance from two levels

Back-solve an unknown source distance y from two levels a known increment dr apart.

Point: $y = dr / (10^{(dL/20)} - 1)$ | Line: $y = dr / (10^{(dL/10)} - 1)$
 $dL = L_1 - L_2$ | (L_1 nearer, L_2 farther by dr)

Use case: 'level is L1 at an unknown distance y; moving dr further gives L2; find y.'

Source file: `acoustics/solveDistance.m`

```

function R = solveDistance(L1, L2, dr)
%SOLVEDISTANCE Source distance from two levels a known increment apart.
% R = ACOUSTICS.SOLVEDISTANCE(L1, L2, dr) where L1 (dB) is measured at an
% unknown distance y and L2 (dB) a further dr (m) away, back-solves y:
% point: y = dr / (10^(dL/20) - 1)
% line: y = dr / (10^(dL/10) - 1), dL = L1 - L2
% R has fields .point (m), .line (m), .dL (dB) and .steps.
arguments
    L1 (1,1) double
    L2 (1,1) double
    dr (1,1) double {mustBePositive}
end
R.dL = L1 - L2;
if ~(R.dL > 0)
    error('acoustics:solveDistance:order', 'Near level L1 must exceed far level L2.');


```

end
Rp = 10^(R.dL/20); Rl = 10^(R.dL/10);
R.point = dr/(Rp - 1);
R.line = dr/(Rl - 1);

```


```

```

R.steps = { ...
    sprintf('dL = L1 - L2 = %.4g - %.4g = %.4g dB', L1, L2, R.dL), ...
    sprintf('Point: y = dr/(10^(dL/20) - 1) = %g/(%.5g - 1) = %.3f m', dr, Rp, R.point), ...
    sprintf('Line: y = dr/(10^(dL/10) - 1) = %g/(%.5g - 1) = %.3f m', dr, Rl, R.line));
end

```

Distance: $L_w \leftrightarrow L_p$ (free field / ground)

Convert between sound power level and SPL at a distance. for point or line sources in various fields.

Point: $L_p = L_w - 20 \cdot \log_{10}(r) - 10 \cdot \log_{10}(4 \cdot \pi / Q)$ | [Q: 1 free field, 2 ground, 4 edge, 8 corner]

Line: | $L_p = L_w - 10 \cdot \log_{10}(r) - k$ | [k = 8 free field, 5 on ground]

Source file: `acoustics/lwlpDistance.m`

```

function R = lwlpDistance(r, opts)
%LWLPDISTANCE Convert between sound power level and SPL at a distance.
% R = ACOUSTICS.LWLPDISTANCE(r, 'Lw', value, 'type', t) gives Lp, and
% R = ACOUSTICS.LWLPDISTANCE(r, 'Lp', value, 'type', t) gives Lw, for a
% source at distance r (m). Supply exactly one of Lw/Lp.
%
% 'type' selects the radiation model (default "point_free"):
% "point_free" point, free field Q=1
% "point_ground" point, on ground Q=2
% "point_edge" point, at an edge Q=4
% "point_corner" point, in a corner Q=8
% "line_free" line, free field
% "line_ground" line, on ground
% Point: Lp = Lw - 20*log10(r) - 10*log10(4*pi/Q)
% Line: Lp = Lw - 10*log10(r) - k (k = 8 free field, 5 on ground)
% R has fields .Lw, .Lp, .coef, .k and .steps.
arguments
    r (1,1) double {mustBePositive}
    opts.Lw double = NaN
    opts.Lp double = NaN
    opts.type (1,1) string {mustBeMember(opts.type, ...
        ["point_free","point_ground","point_edge","point_corner", ...
        "line_free","line_ground"])} = "point_free"
end
hasLw = ~isnan(opts.Lw);
hasLp = ~isnan(opts.Lp);
if hasLw == hasLp
    error('acoustics:lwlpDistance:input', 'Supply exactly one of Lw or Lp.');
```

```

end
switch opts.type
    case "point_free", coef = 20; k = 10*log10(4*pi/1);
    case "point_ground", coef = 20; k = 10*log10(4*pi/2);
    case "point_edge", coef = 20; k = 10*log10(4*pi/4);
    case "point_corner", coef = 20; k = 10*log10(4*pi/8);
    case "line_free", coef = 10; k = 8;
    otherwise, coef = 10; k = 5; % line_ground
end
R.coef = coef; R.k = k;
if hasLw
    R.Lw = opts.Lw;
    R.Lp = R.Lw - coef*log10(r) - k;
    R.steps = { ...
        sprintf('Lp = Lw - %g*log10(r) - %.2f', coef, k), ...
        sprintf('= %.4g - %g*log10(%g) - %.2f = %.2f dB', ...
            R.Lw, coef, r, k, R.Lp)};
else
    R.Lp = opts.Lp;
    R.Lw = R.Lp + coef*log10(r) + k;
    R.steps = { ...
        sprintf('Lw = Lp + %g*log10(r) + %.2f', coef, k), ...
        sprintf('= %.4g + %g*log10(%g) + %.2f = %.2f dB', ...
            R.Lp, coef, r, k, R.Lw)};
end
end

```

Room Acoustics

Room: Sabine reverberation time

Solve the Sabine equation for whichever ONE of V, S, alpha, T60 is unknown.

$$T60 = 0.161 * V / (\alpha * S) \mid A = \alpha * S \mid (\text{total absorption, m}^2)$$

Source file: +acoustics/sabineT60.m

```
function R = sabineT60(opts)
%SABINET60 Sabine reverberation time, solving for any one unknown.
% R = ACOUSTICS.SABINET60('V',...,'S',...,'alpha',...,'T60',...) solves the
% Sabine equation for whichever ONE of the four is left out (NaN/omitted):
% T60 = 0.161 * V / (alpha * S)
% Supply exactly three of V (m^3), S (m^2), alpha, T60 (s). R has fields
% .V, .S, .alpha, .T60, .A (= alpha*S) and .steps.
%
% Example:
% acoustics.sabineT60('V',200,'S',240,'alpha',0.15).T60
arguments
    opts.V      double = NaN
    opts.S      double = NaN
    opts.alpha  double = NaN
    opts.T60    double = NaN
end
V = opts.V; S = opts.S; a = opts.alpha; T = opts.T60;
miss = isnan(V) + isnan(S) + isnan(a) + isnan(T);
if miss ~= 1
    error('acoustics:sabineT60:input', ...
        'Supply exactly three of V, S, alpha, T60 (leave one out).');
end
if isnan(T), T = 0.161*V/(a*S);
elseif isnan(a), a = 0.161*V/(T*S);
elseif isnan(S), S = 0.161*V/(T*a);
else, V = T*a*S/0.161;
end
R.V = V; R.S = S; R.alpha = a; R.T60 = T; R.A = a*S;
R.steps = { ...
    'T60 = 0.161*V / (alpha*S)', ...
    sprintf(' = 0.161*%.4g / (%.4f*%.4g) = %.3f / %.3f = %.3f s', ...
        V, a, S, 0.161*V, a*S, T), ...
    sprintf('Absorption A = alpha*S = %.2f m^2', R.A)};
end
```

Room: average absorption coefficient

Area-weighted mean absorption coefficient over several surfaces.

$$\alpha_{\text{bar}} = \frac{\sum(\alpha_i * S_i)}{\sum(S_i)}$$

Source file: +acoustics/averageAbsorption.m

```
function R = averageAbsorption(areas, alphas)
%AVERAGEABSORPTION Area-weighted mean absorption coefficient.
% R = ACOUSTICS.AVERAGEABSORPTION(areas, alphas) with equal-length vectors
% of surface areas (m^2) and their absorption coefficients returns:
% alpha_bar = sum(alpha_i * S_i) / sum(S_i)
% R has fields .alphaBar, .totalArea (m^2), .totalAbsorption (m^2) and .steps.
arguments
    areas (1,:) double {mustBeNonnegative}
    alphas (1,:) double {mustBeNonnegative}
end
if numel(areas) ~= numel(alphas)
    error('acoustics:averageAbsorption:size', 'areas and alphas must match.');
```

```
end
num = sum(areas .* alphas);
den = sum(areas);
if den == 0
    error('acoustics:averageAbsorption:area', 'Total area is zero.');
```

```
end
R.alphaBar = num/den;
R.totalArea = den;
R.totalAbsorption = num;
R.steps = { ...
    'alpha_bar = sum(alpha_i*S_i) / sum(S_i)', ...
    sprintf(' = %.3f / %.1f = %.4f', num, den, R.alphaBar)};
end
```

Room: room constant R

Room constant from average absorption and total surface area.

$$R = \alpha * S / (1 - \alpha)$$

Source file: +acoustics/roomConstant.m

```
function R = roomConstant(alpha, S)
%ROOMCONSTANT Room constant from average absorption and surface area.
```

```
% R = ACOUSTICS.ROOMCONSTANT(alpha, S) returns
% R = alpha*S / (1 - alpha)
% for average absorption alpha (0 < alpha < 1) and total surface S (m^2).
% R has fields .R (m^2) and .steps.
arguments
    alpha (1,1) double
    S      (1,1) double {mustBePositive}
end
if ~(alpha > 0 && alpha < 1)
    error('acoustics:roomConstant:alpha', 'alpha must be between 0 and 1.');
```

Room: room equation Lp from Lw

SPL at a distance from a source, combining the direct and reverberant fields.

$$L_p = L_w + 10 \cdot \log_{10} \left(\frac{Q}{4 \cdot \pi \cdot r^2} + \frac{4}{R} \right)$$

Source file: `+acoustics/roomEquation.m`

```
function R = roomEquation(Lw, r, Rc, opts)
%ROOMEQUATION SPL in a room from sound power (direct + reverberant field).
% R = ACOUSTICS.ROOMEQUATION(Lw, r, Rc) returns the SPL at distance r (m)
% from a source of power level Lw (dB) in a room of room constant Rc (m^2):
% Lp = Lw + 10*log10( Q/(4*pi*r^2) + 4/R )
% Optional 'Q' (default 1) sets the directivity. R has fields .Lp (dB),
% .direct, .reverberant (the two bracket terms), .dominant and .steps.
arguments
    Lw (1,1) double
    r   (1,1) double {mustBePositive}
    Rc (1,1) double {mustBePositive}
    opts.Q (1,1) double {mustBePositive} = 1
end
R.direct = opts.Q/(4*pi*r^2);
R.reverberant = 4/Rc;
R.Lp = Lw + 10*log10(R.direct + R.reverberant);
if R.direct > R.reverberant
    R.dominant = 'direct field dominates';
else
    R.dominant = 'reverberant field dominates';
end
R.steps = { ...
    'Lp = Lw + 10*log10( Q/(4*pi*r^2) + 4/R )', ...
    sprintf(' = %.4g + 10*log10( %.4g + %.4g )', Lw, R.direct, R.reverberant), ...
    sprintf(' = %.4g + 10*log10( %.4g ) = %.2f dB (%s)', ...
        Lw, R.direct + R.reverberant, R.Lp, R.dominant)};
end
```

Room: reverberant change (add/remove panels)

Change in reverberant SPL when a patch of absorber is added to or removed from a room.

$$A_1 = 0.161 \cdot V / T_{60} \quad | \quad (\text{existing absorption})$$

$$A_{\text{abs}} = S_{\text{abs}} \cdot \alpha \quad | \quad (\text{absorber contribution})$$

$$A_2 = A_1 \pm A_{\text{abs}} \quad | \quad (\text{new absorption; + add, - remove})$$

$$\Delta L_p = 10 \cdot \log_{10}(A_1 / A_2) \quad | \quad (\text{negative when adding absorber})$$

Use case: predict the SPL rise/drop when acoustic panels are added or removed from a room.

Source file: `+acoustics/absorberChange.m`

```
function R = absorberChange(V, T60, Sabs, alpha, opts)
%ABSORBERCHANGE Reverberant-field level change from adding/removing absorber.
% R = ACOUSTICS.ABSORBERCHANGE(V, T60, Sabs, alpha) computes the change in
% reverberant SPL in a room of volume V (m^3) whose current reverberation
% time is T60 (s) when a patch of absorber of area Sabs (m^2) and
% coefficient alpha is added (default) or removed:
% A1 = 0.161*V / T60      existing absorption
% A_abs = Sabs * alpha    absorber contribution
% A2 = A1 +/- A_abs      new absorption
% dLp = 10*log10(A1 / A2) level change (negative when adding)
% Use 'mode','remove' to remove absorber (A2 = A1 - A_abs). R has fields
% .A1, .Aabs, .A2 (m^2), .deltaLp (dB) and .steps.
arguments
    V      (1,1) double {mustBePositive}
```

```

T60 (1,1) double {mustBePositive}
Sabs (1,1) double {mustBePositive}
alpha (1,1) double {mustBeNonnegative}
opts.mode (1,1) string {mustBeMember(opts.mode,["add","remove"])} = "add"
end
R.A1 = 0.161*V/T60;
R.Aabs = Sabs*alpha;
if opts.mode == "remove"
    R.A2 = R.A1 - R.Aabs; sign = '-';
else
    R.A2 = R.A1 + R.Aabs; sign = '+';
end
if ~(R.A2 > 0)
    error('acoustics:absorberChange:A2', ...
        'Absorber exceeds room absorption (A2 <= 0).');
end
R.deltaLp = 10*log10(R.A1/R.A2);
R.steps = { ...
    sprintf('A1 = 0.161*V/T60 = 0.161*g/%g = %.3f m^2', V, T60, R.A1), ...
    sprintf('A_abs = Sabs*alpha = %g*g = %.3f m^2', Sabs, alpha, R.Aabs), ...
    sprintf('A2 = A1 %s A_abs = %.3f m^2', sign, R.A2), ...
    sprintf('dLp = 10*log10(A1/A2) = 10*log10(%.3f/%.3f) = %+.2f dB', ...
        R.A1, R.A2, R.deltaLp)};
end

```

Room: plant room (surface treatment)

Reverberant-field level in a plant room with several machines, before/after a surface coating.

Per octave band: $L_w = 10 \cdot \log_{10}(\sum 10^{(L_{w_machine}/10)})$ | (combine machines)

Before: $\alpha_{bar} = \alpha_{Base}$

After: | $\alpha_{bar} = [S_{coat} \cdot \alpha_{Coat} + (S - S_{coat}) \cdot \alpha_{Base}] / S$

$R_{room} = S \cdot \alpha_{bar} / (1 - \alpha_{bar})$ | $L_p = L_w + 10 \cdot \log_{10}(4/R_{room})$

Overall dB(A) = energy-sum of (L_p + A-weighting) across bands; reduction = before - after

Use case: motors in a plant room; combine their L_w per band, get reverberant SPL, then treat a surface.

Source file: `+acoustics/plantRoom.m`

```

function R = plantRoom(freqs, machineLw, alphaBase, alphaCoat, Scoat, S, opts)
%PLANTROOM Reverberant-field level in a plant room, before/after coating.
% R = ACOUSTICS.PLANTROOM(freqs, machineLw, alphaBase, alphaCoat, Scoat, S)
% works per octave band to size a surface-absorption treatment.
%
% Inputs (per octave band unless noted):
%   freqs      1xB octave centre frequencies (Hz)
%   machineLw  BxM matrix of machine sound power levels (dB); the M
%               machines in each band are combined on an energy basis
%   alphaBase  1xB bare-surface absorption coefficient
%   alphaCoat  1xB absorption coefficient of the coated surface
%   Scoat      area of the surface being coated (m^2, scalar)
%   S          total room surface area (m^2, scalar)
%
% For each band:
%   Lw = 10*log10( sum 10^(Lw_machine/10) )    combined power level
%   W = W_ref * 10^(Lw/10)
%   BEFORE: alpha_bar = alphaBase
%   AFTER:  alpha_bar = [Scoat*alphaCoat + (S-Scoat)*alphaBase] / S
%   R_room = S*alpha_bar/(1-alpha_bar)
%   Lp = Lw + 10*log10(4/R_room)                reverberant field
% Overall A-weighted levels (default 'net','A') are formed by energy-summing
% the per-band (Lp + weighting), before and after; the reduction is their
% difference.
%
% R has fields:
%   .freqs, .Lw (1xB), .W (1xB), .LwOverall
%   .alphaAfter (1xB), .Rbefore (1xB), .Rafter (1xB)
%   .LpBefore (1xB), .LpAfter (1xB)
%   .dBAbefore, .dBAafter, .reduction
%   .steps
arguments
freqs (1,:) double {mustBePositive}
machineLw (:,:) double
alphaBase (1,:) double {mustBePositive}
alphaCoat (1,:) double {mustBePositive}
Scoat (1,1) double {mustBePositive}
S (1,1) double {mustBePositive}
opts.net (1,1) char {mustBeMember(opts.net,{'A','B','C','Z'})} = 'A'
end
C = acoustics.constants();

```

```

B = numel(freqs);
if size(machineLw,1) ~= B
    error('acoustics:plantRoom:rows', 'machineLw must have one row per band.');
```

```

end
if numel(alphaBase) ~= B || numel(alphaCoat) ~= B
    error('acoustics:plantRoom:size', 'alphaBase/alphaCoat must have one value per band.');
```

```

end
if Scoat > S
    error('acoustics:plantRoom:area', 'Coated area cannot exceed total surface.');
```

```

end

R.freqs = freqs;
R.Lw = 10*log10(sum(10.^(machineLw/10), 2));          % combine machines
R.W = C.WREF * 10.^(R.Lw/10);
R.LwOverall = 10*log10(sum(10.^(R.Lw/10)));

R.alphaAfter = (Scoat*alphaCoat + (S-Scoat)*alphaBase) / S;
R.Rbefore = S*alphaBase ./ (1 - alphaBase);
R.Rafter = S*R.alphaAfter ./ (1 - R.alphaAfter);
R.LpBefore = R.Lw + 10*log10(4 ./ R.Rbefore);
R.LpAfter = R.Lw + 10*log10(4 ./ R.Rafter);

w = arrayfun(@(f) acoustics.weightingValue(f, opts.net), freqs);
R.dBAbefore = 10*log10(sum(10.^((R.LpBefore + w)/10)));
R.dBAafter = 10*log10(sum(10.^((R.LpAfter + w)/10)));
R.reduction = R.dBAbefore - R.dBAafter;

tag = 'dB'; if opts.net ~= 'Z', tag = sprintf('dB(%c)', opts.net); end
lines = { ...
    'Per band: Lw = 10*log10( sum 10^(Lw_machine/10) ), W = W_ref*10^(Lw/10)', ...
    'R = S*alpha_bar/(1-alpha_bar), Lp = Lw + 10*log10(4/R)', ...
    'after: alpha_bar = [Scoat*alphaCoat + (S-Scoat)*alphaBase]/S', ''};
for i = 1:B
    lines[end+1] = sprintf(' %6g Hz: Lw %.1f (W %.4g) Lp %.1f -> %.1f dB', ...
        freqs(i), R.Lw(i), R.W(i), R.LpBefore(i), R.LpAfter(i)); %#ok<AGROW>
end
R.steps = [lines, { '', ...
    sprintf('Overall Lw = %.2f dB', R.LwOverall), ...
    sprintf('Overall before = %.1f %s . after = %.1f %s . reduction = %.1f %s', ...
        R.dBAbefore, tag, R.dBAafter, tag, R.reduction, tag)]];
end

```

Room: reverberation test room (mean-square p)

Reverberant mean-square pressure in a test room. empty vs furnished (exact impedance form).

$$A = 0.161 \cdot V / T60 \quad | \quad \alpha = A/S \quad | \quad R_{\text{room}} = A/(1-\alpha)$$

$$W = W_{\text{ref}} \cdot 10^{(Lw/10)}$$

$$\langle p^2 \rangle = 4 \cdot \rho \cdot c \cdot W / R_{\text{room}} \quad | \quad Lp = 10 \cdot \log_{10}(\langle p^2 \rangle / p_{\text{ref}}^2)$$

Source file: `acoustics/reverbTestRoom.m`

```

function R = reverbTestRoom(freqs, Lw, T60empty, T60furnished, V, S, opts)
%REVERBTSTROOM Reverberant mean-square pressure in a test room.
% R = ACOUSTICS.REVERBTSTROOM(freqs, Lw, T60empty, T60furnished, V, S)
% evaluates the reverberant field for an empty and a furnished state using
% the EXACT characteristic impedance (not the dB approximation):
%   A      = 0.161*V / T60      room absorption (m^2)
%   alpha  = A / S
%   R_room = A / (1 - alpha)
%   W      = W_ref * 10^(Lw/10)
%   <p^2>   = 4 * rho * c * W / R_room  reverberant mean-square pressure (Pa^2)
%   Lp     = 10*log10( <p^2> / p_ref^2 )
%
% Inputs are per octave band (1xB vectors) except V, S:
%   freqs   octave centre frequencies (Hz)
%   Lw      band sound power levels (dB re 1e-12 W)
%   T60empty reverberation time, empty room (s)
%   T60furnished reverberation time, furnished room (s)
%   V, S     room volume (m^3) and total surface (m^2)
% Optional: 'rhoc' (default 415 rayls), 'net' (default 'A').
%
% R has fields (all per-band vectors are 1xB):
%   .freqs, .W      absorption area (m^2)
%   .Aempty, .Afurn average absorption coefficient
%   .alphaEmpty, .alphaFurn
%   .p2empty, .p2furn mean-square pressure (Pa^2)
%   .LpEmpty, .LpFurn band SPL, unweighted (dB)
%   .LpAempty, .LpAfurn band SPL, weighted (dB, per 'net')
%   .dBAempty, .dBAfurn, .reduction overall weighted levels
%   .steps

```

```

arguments
    freqs      (1,:) double {mustBePositive}
    Lw         (1,:) double
    T60empty   (1,:) double {mustBePositive}
    T60furnished (1,:) double {mustBePositive}
    V (1,1) double {mustBePositive}
    S (1,1) double {mustBePositive}
    opts.rhoc (1,1) double {mustBePositive} = 415
    opts.net   (1,1) char {mustBeMember(opts.net,{'A','B','C','Z'})} = 'A'
end
C = acoustics.constants();
B = numel(freqs);
if ~isequal(numel(Lw), numel(T60empty), numel(T60furnished), B)
    error('acoustics:reverbTestRoom:size', 'All band vectors must match freqs.');
```

```

end
W = C.WREF * 10.^(Lw/10);

[R.Aempty, R.p2empty, R.LpEmpty] = state(T60empty, W, V, S, opts.rhoc, C.PREF);
[R.Afurn, R.p2furn, R.LpFurn] = state(T60furnished, W, V, S, opts.rhoc, C.PREF);

R.freqs = freqs;
R.W = W;
R.alphaEmpty = R.Aempty / S;
R.alphaFurn = R.Afurn / S;
w = arrayfun(@(f) acoustics.weightingValue(f, opts.net), freqs);
R.LpAempty = R.LpEmpty + w;
R.LpAfurn = R.LpFurn + w;
R.dBAempty = 10*log10(sum(10.^(R.LpAempty/10)));
R.dBAfurn = 10*log10(sum(10.^(R.LpAfurn/10)));
R.reduction = R.dBAempty - R.dBAfurn;

tag = 'dB'; if opts.net ~= 'Z', tag = sprintf('dB(%c)', opts.net); end
R.steps = { ...
    'REVERBERATION TEST ROOM (per octave band)', ...
    'A = 0.161*V/T60 * alpha = A/S * R = A/(1-alpha)', ...
    'W = W_ref*10^(Lw/10) * <p^2> = 4*rho c*W/R * Lp = 10*log10(<p^2>/p_ref^2)', ...
    '', ...
    ['(a) A, empty room (m^2):' ' rowStr(freqs, R.Aempty, '%.3f')'], ...
    ['(b) A, room + furniture (m^2):' ' rowStr(freqs, R.Afurn, '%.3f')'], ...
    ['(c) alpha-bar, empty:' ' rowStr(freqs, R.alphaEmpty, '%.4f')'], ...
    ['(d) alpha-bar, furnished:' ' rowStr(freqs, R.alphaFurn, '%.4f')'], ...
    ['(e) sound power W (W):' ' rowStr(freqs, R.W, '%.4g')'], ...
    ['(f) <p^2>, empty (Pa^2):' ' rowStr(freqs, R.p2empty, '%.4g')'], ...
    ['(g) <p^2>, furnished (Pa^2):' ' rowStr(freqs, R.p2furn, '%.4g')'], ...
    ['(h) Lp, empty (' tag '):' ' rowStr(freqs, R.LpAempty, '%.2f')'], ...
    ['(i) Lp, furnished (' tag '):' ' rowStr(freqs, R.LpAfurn, '%.2f')'], ...
    '', ...
    sprintf('(j) Overall empty = %.1f %s', R.dBAempty, tag), ...
    sprintf('(k) Overall furnished = %.1f %s', R.dBAfurn, tag), ...
    sprintf('(l) Reduction = %.1f %s', R.reduction, tag)};
end

function s = rowStr(freqs, vals, fmt)
    parts = arrayfun(@(j) sprintf(['%g Hz ' fmt], freqs(j), vals(j)), ...
        1:numel(freqs), 'UniformOutput', false);
    s = strjoin(parts, ' ');
end

function [A, p2, Lp] = state(T60, W, V, S, rhoc, pref)
    A = 0.161*V ./ T60;
    alpha = A / S;
    Rroom = A ./ (1 - alpha);
    p2 = 4*rhoc*W ./ Rroom;
    Lp = 10*log10(p2 / pref^2);
end

```

Sound Power Measurement

Power: background correction K1

Background-noise correction for a sound power measurement (requires $dL > 6$ dB).

$$dL = L_{\text{source}} - L_{\text{background}}$$

$$K1 = -10 \cdot \log_{10}(1 - 10^{(-dL/10)}) \quad | \quad [\text{negligible when } dL \geq 15 \text{ dB}]$$

Source file: `+acoustics/backgroundK1.m`

```

function R = backgroundK1(Lsource, Lbackground)
%BACKGROUNDK1 Background-noise correction K1 for sound power measurement.

```

```
% R = ACOUSTICS.BACKGROUNDK1(Lsource, Lbackground) with the mean SPL with
% the source running and the mean background SPL (dB) returns:
% dL = Lsource - Lbackground
% K1 = -10*log10( 1 - 10^(-dL/10) )
% Errors when dL <= 6 dB (measurement invalid). R has fields .dL (dB),
% .K1 (dB), .negligible (logical, dL >= 15) and .steps.
arguments
    Lsource (1,1) double
    Lbackground (1,1) double
end
R.dL = Lsource - Lbackground;
if R.dL < 6
    error('acoustics:backgroundK1:invalid', ...
        'dL = %.1f dB < 6 dB - background too high, measurement invalid.', R.dL);
end
R.K1 = -10*log10(1 - 10^(-R.dL/10));
R.negligible = R.dL >= 15;
note = '';
if R.negligible, note = ' (>=15 dB -> negligible)'; end
R.steps = { ...
    'K1 = -10*log10(1 - 10^(-dL/10))', ...
    sprintf('dL = %.1f dB', R.dL), ...
    sprintf(' = -10*log10(1 - 10^(-%.1f/10)) = %.3f dBs', R.dL, R.K1, note)};
end
```

Power: environmental correction K2

Room-reflection correction from the measurement surface area and the room's absorption area.

$$K2 = 10 \cdot \log_{10}(1 + 4 \cdot S/A)$$

Source file: `+acoustics/environmentalk2.m`

```
function R = environmentalK2(S, A)
%ENVIRONMENTALK2 Environmental (room reflection) correction K2.
% R = ACOUSTICS.ENVIRONMENTALK2(S, A) with measurement surface S (m^2) and
% room absorption area A (m^2) returns:
% K2 = 10*log10( 1 + 4*S/A )
% R has fields .K2 (dB) and .steps.
arguments
    S (1,1) double {mustBePositive}
    A (1,1) double {mustBePositive}
end
R.K2 = 10*log10(1 + 4*S/A);
R.steps = { ...
    'K2 = 10*log10(1 + 4S/A)', ...
    sprintf(' = 10*log10(1 + 4*%g/%g) = %.3f dB', S, A, R.K2)};
end
```

Power: sound power level (measured)

Sound power level from a mean measurement-surface SPL, with K1/K2 corrections.

$$Lw = (Lp - K1 - K2) + 10 \cdot \log_{10}(S / S0), \quad | \quad S0 = 1 \text{ m}^2$$

Source file: `+acoustics/soundPowerMeasured.m`

```
function R = soundPowerMeasured(Lp, S, opts)
%SOUNDPOWERMEASURED Sound power level from a mean surface SPL.
% R = ACOUSTICS.SOUNDPOWERMEASURED(Lp, S) with the mean measurement-surface
% SPL Lp (dB) over an area S (m^2) returns the sound power level:
% Lw = (Lp - K1 - K2) + 10*log10(S/S0), S0 = 1 m^2
% Optional 'K1' and 'K2' (default 0) apply the background and environmental
% corrections. R has fields .Lw (dB) and .steps.
arguments
    Lp (1,1) double
    S (1,1) double {mustBePositive}
    opts.K1 (1,1) double = 0
    opts.K2 (1,1) double = 0
end
R.Lw = (Lp - opts.K1 - opts.K2) + 10*log10(S);
R.steps = { ...
    'Lw = (Lp - K1 - K2) + 10*log10(S/S0), S0 = 1 m^2', ...
    sprintf(' = (%g - %g - %g) + 10*log10(%g) = %.2f dB', ...
        Lp, opts.K1, opts.K2, S, R.Lw)};
end
```

Power: Lw from enveloping-surface SPLs

Sound power level from SPLs measured on the partial areas of an enveloping surface (e.g. a cuboid).


```
Lw = 10*log10( sum S_i * 10^(Li/10) ) | [S0 = 1 m^2]
    = meanLp + 10*log10(S), | meanLp = 10*log10( (1/S) sum S_i*10^(Li/10) )
```

Use case: octave-band SPLs measured on each face of an imaginary box around a machine.

Source file: `+acoustics/soundPowerFromSurfaces.m`

```
function R = soundPowerFromSurfaces(levels, areas)
%SOUNDPOWERFROMSURFACES Sound power level from partial measurement surfaces.
% R = ACOUSTICS.SOUNDPOWERFROMSURFACES(levels, areas) determines the sound
% power level of a source from the sound pressure levels measured on the
% partial areas of an enveloping surface (e.g. an imaginary cuboid around a
% machine):
%   Lw = 10*log10( sum S_i * 10^(Li/10) )      [S0 = 1 m^2]
%   = meanLp + 10*log10(S)
% where meanLp = 10*log10( (1/S) sum S_i 10^(Li/10) ) is the area-weighted
% mean SPL and S = sum S_i the total surface area.
%
% LEVELS are the per-surface SPLs (dB); use A-weighted levels to get the
% A-weighted Lw. AREAS are the per-surface areas (m^2); pass a scalar if all
% partial areas are equal.
%
% R has fields .totalArea (m^2), .meanLp (dB), .Lw (dB) and .steps.
%
% Example (six 2 m^2 faces, A-weighted SPLs):
%   acoustics.soundPowerFromSurfaces([89 87 80 95 90 87], 2).Lw   % 100.8
arguments
    levels (1,:) double
    areas  (1,:) double {mustBePositive}
end
if isscalar(areas)
    areas = repmat(areas, 1, numel(levels));
end
if numel(areas) ~= numel(levels)
    error('acoustics:soundPowerFromSurfaces:size', ...
        'Give one area per level (or a single area for all).');
end
e = areas .* 10.^(levels/10);
R.totalArea = sum(areas);
R.meanLp = 10*log10(sum(e)/R.totalArea);
R.Lw = 10*log10(sum(e));
R.steps = { ...
    'Lw = 10*log10( sum S_i*10^(Li/10) ) = meanLp + 10*log10(S), S0 = 1 m^2', ...
    sprintf('S = sum S_i = %.4g m^2', R.totalArea), ...
    sprintf('meanLp = 10*log10( (1/S) sum S_i*10^(Li/10) ) = %.2f dB', R.meanLp), ...
    sprintf('Lw = %.2f + 10*log10(%.4g) = %.2f dB', R.meanLp, R.totalArea, R.Lw)};
end
```

Power: Lw from free-field band SPLs

Free-field sound power level from measured octave-band SPLs over a hemisphere/measurement surface.

```
L_lin,i = Li - Wi | (remove weighting)
Lp = 10*log10( sum 10^(L_lin,i/10) )
I = p_ref^2 * 10^(Lp/10) / (rho c) | W = I*S, | Lw = 10*log10(W/W_ref)
```

Use case: octave-band A-weighted SPLs at several points on a hemisphere around a source (e.g. a drill).

Source file: `+acoustics/lwFromBands.m`

```
function R = lwFromBands(freqs, levels, S, opts)
%LWFROMBANDS Free-field sound power level from measured band SPLs.
% R = ACOUSTICS.LWFROMBANDS(freqs, levels, S) un-weights the band levels
% (default 'net','A'), energy-sums them to an overall linear SPL, converts
% to intensity and integrates over the measurement surface S (m^2):
%   L_lin,i = Li - Wi      (remove weighting)
%   Lp      = 10*log10( sum 10^(L_lin,i/10) )
%   I       = p_ref^2 * 10^(Lp/10) / (rho c)
%   W       = I * S,   Lw = 10*log10(W / W_ref)
% 'net' is the weighting applied to the input levels ('A','B','C','Z').
% R has fields .Lp (dB), .I (W/m^2), .W (W), .Lw (dB) and .steps.
arguments
    freqs (1,:) double {mustBePositive}
    levels (1,:) double
    S      (1,1) double {mustBePositive}
    opts.net (1,1) char {mustBeMember(opts.net,{'A','B','C','Z'})} = 'A'
end
if numel(freqs) ~= numel(levels)
    error('acoustics:lwFromBands:size', 'freqs and levels must match.');
```

```

lin = levels - w;
R.Lp = 10*log10(sum(10.^(lin/10)));
p2 = C.PREF^2 * 10^(R.Lp/10);
R.I = p2/C.RHOC;
R.W = R.I * S;
R.Lw = 10*log10(R.W/C.WREF);
R.steps = { ...
    sprintf('Un-weight bands (remove %c), overall Lp = 10*log10( sum 10^(L_lin/10) ) = %.2f dB', ...
        opts.net, R.Lp), ...
    sprintf('p2_rms = p_ref^2 * 10^(Lp/10) = %.4g Pa^2', p2), ...
    sprintf('I = p2/(rho c) = %.4g W/m^2', R.I), ...
    sprintf('W = I*S = %.4g W', R.W), ...
    sprintf('Lw = 10*log10(W/1e-12) = %.1f dB', R.Lw)};
end

```

Duct / Pipe

Duct: sound power -> mic voltage

Microphone voltage from a duct's sound power (plane-wave). given the mic's V/Pa sensitivity.

$W = W_{ref} \cdot 10^{(Lw/10)}$ | $A = \pi \cdot d^2 / 4$ | $I = W/A$
 $p_{rms} = \sqrt{I \cdot \rho \cdot c}$ | $V = p_{rms} \cdot 10^{(sens/20)}$
 First higher-order mode cut-on: | $f_c = 1.8412 \cdot c / (\pi \cdot d)$

Source file: `+acoustics/ductToVoltage.m`

```

function R = ductToVoltage(Lw, d_mm, sens_dB, opts)
%DUCTTOVOLTAGE Microphone voltage from duct sound power (plane-wave).
% R = ACOUSTICS.DUCTTOVOLTAGE(Lw, d_mm, sens_dB) for a circular duct of
% diameter d_mm (mm) carrying sound power level Lw (dB) with a flush
% microphone of sensitivity sens_dB (dB re 1 V/Pa):
% W = W_ref*10^(Lw/10), A = pi*d^2/4, I = W/A
% p_rms = sqrt(I*rho*c), V = p_rms * 10^(sens/20)
% Also reports the first higher-order mode cut-on f_c = 1.8412*c/(pi*d).
% Optional 'rho' (1.21 kg/m^3), 'c' (343 m/s), 'fmax' (Hz, for a validity
% check). R has fields .W, .area, .I, .prms, .spl, .V, .fcuton and .steps.
arguments
    Lw      (1,1) double
    d_mm    (1,1) double {mustBePositive}
    sens_dB (1,1) double
    opts.rho (1,1) double {mustBePositive} = 1.21
    opts.c   (1,1) double {mustBePositive} = 343
    opts.fmax (1,1) double = 0
end
C = acoustics.constants();
d = d_mm/1000;
R.W = C.WREF*10^(Lw/10);
R.area = pi*d^2/4;
R.I = R.W/R.area;
rc = opts.rho*opts.c;
R.prms = sqrt(R.I*rc);
R.spl = 20*log10(R.prms/C.PREF);
sens = 10^(sens_dB/20);
R.V = R.prms*sens;
R.fcuton = 1.8412*opts.c/(pi*d);
if opts.fmax > 0
    if opts.fmax < R.fcuton
        note = sprintf('Highest freq %.0f Hz < cut-on %.0f Hz -> plane waves only, valid.', ...
            opts.fmax, R.fcuton);
    else
        note = sprintf('Highest freq %.0f Hz >= cut-on %.0f Hz -> higher-order modes, approximate.', ...
            opts.fmax, R.fcuton);
    end
else
    note = sprintf('First higher-order mode cuts on at %.0f Hz (plane-wave assumption valid below).', ...
        R.fcuton);
end
R.steps = { ...
    sprintf('W = W_ref*10^(Lw/10) = %.4g W', R.W), ...
    sprintf('A = pi*d^2/4 = %.4g m^2', R.area), ...
    sprintf('I = W/A = %.4g W/m^2', R.I), ...
    sprintf('p_rms = sqrt(I*rho*c) = %.4g Pa (SPL = %.1f dB)', R.prms, R.spl), ...
    sprintf('V = p_rms*10^(S/20) = %.4g V (%.4g mV)', R.V, R.V*1000), ...
    note};
end

```

Duct: octave-band Lp -> I, W, totals (pipe)

Full per-band chain (p_{rms} , I , W) plus overall L_p/L_w totals for a long reflection-free pipe.

```
p_rms = p_ref * 10^(Lp/20) | I = p_rms^2 / (rho * c)
W = I * A, | A = pi*d^2/4 | (pipe cross-section)
Lp_total = 10*log10( sum 10^(Lp/10) ) | Lw_total = 10*log10( sum W / W_ref )
```

Use case: speaker in a long pipe, no reflections; octave-band SPLs given -> find p_{rms} , I , W per band, totals.

Source file: `acoustics/ductBandPower.m`

```
function R = ductBandPower(freqs, Lp, d, opts)
%DUCTBANDPOWER Plane-wave sound power radiated from a long pipe/duct.
% R = ACOUSTICS.DUCTBANDPOWER(freqs, Lp, d) for a long pipe (no reflected
% waves) of internal diameter d (m), given the octave-band sound pressure
% levels Lp (dB), computes per band:
% (i) p_rms = p_ref * 10^(Lp/20) (Pa)
% (ii) I = p_rms^2 / (rho * c) (W/m^2)
% (iii) W = I * A, A = pi*d^2/4 (W) [pipe cross-section]
% and the totals:
% (iv) Lp_total = 10*log10( sum 10^(Lp/10) ) (dB)
% (v) Lw_total = 10*log10( sum W / W_ref ) (dB re 1e-12 W)
%
% Uses the EXACT rho * c = rho*c (defaults rho = 1.21 kg/m^3, c = 343 m/s;
% override with 'rho' / 'c'). d is in metres (e.g. 0.086 for an 86 mm pipe).
%
% R has fields .prms, .I, .W (1xB), .area, .LpTotal, .Wtotal, .LwTotal, .steps.
%
% Example (86 mm pipe):
% R = acoustics.ductBandPower([125 250 500 1000], [106 105 105 94], 0.086);
% R.LwTotal % 87.72 dB
arguments
freqs (1,:) double {mustBePositive}
Lp (1,:) double
d (1,1) double {mustBePositive}
opts.rho (1,1) double {mustBePositive} = 1.21
opts.c (1,1) double {mustBePositive} = 343
end
if numel(freqs) ~= numel(Lp)
    error('acoustics:ductBandPower:size', 'freqs and Lp must match.');
end
C = acoustics.constants();
rhoc = opts.rho*opts.c;
R.area = pi*d^2/4;
R.prms = C.PREF * 10.^(Lp/20); % (i)
R.I = R.prms.^2 / rhoc; % (ii)
R.W = R.I * R.area; % (iii)
R.LpTotal = 10*log10(sum(10.^(Lp/10))); % (iv)
R.Wtotal = sum(R.W);
R.LwTotal = 10*log10(R.Wtotal/C.WREF); % (v)

R.steps = { ...
    'LONG PIPE / DUCT, plane waves (no reflected waves)', ...
    sprintf('rho c = %.2f rayls · A = pi*d^2/4 = %.5g m^2', rhoc, R.area), ...
    'p_rms = p_ref*10^(Lp/20) · I = p_rms^2/(rho c) · W = I*A', ...
    '', ...
    ['(i) p_rms (Pa):' ' rowStr(freqs, R.prms, '%.3f')'], ...
    ['(ii) I (W/m^2):' ' rowStr(freqs, R.I, '%.4g')'], ...
    ['(iii) W (W):' ' rowStr(freqs, R.W, '%.4g')'], ...
    '', ...
    sprintf('(iv) Total Lp = 10*log10(sum 10^(Lp/10)) = %.2f dB', R.LpTotal), ...
    sprintf('(v) Total Lw = 10*log10(sum W / 1e-12) = %.2f dB', R.LwTotal)};
end

function s = rowStr(freqs, vals, fmt)
parts = arrayfun(@(j) sprintf(['%g Hz ' fmt], freqs(j), vals(j)), ...
    1:numel(freqs), 'UniformOutput', false);
s = strjoin(parts, ' ');
end
```

Weighting & Band Workbench

A / B / C Weighting & overall level

Apply A/B/C/Z frequency weighting to a band spectrum and energy-sum to an overall level.

```
L_weighted = 10*log10( sum 10^((Li + Wi)/10) )
L_linear (unweighted) = 10*log10( sum 10^(Li/10) )
```

Source file: +acoustics/weightedOverall.m

```
function R = weightedOverall(freqs, levels, net)
%WEIGHTEDOVERALL Overall weighted and linear level of a band spectrum.
% R = ACOUSTICS.WEIGHTEDOVERALL(freqs, levels, net) applies the A/B/C/Z
% weighting NET to octave or 1/3-octave band levels (dB) and energy-sums:
% L_W = 10*log10( sum 10^((Li + Wi)/10) )
% linear (unweighted) = 10*log10( sum 10^(Li/10) )
% R has fields .weighted (dB), .linear (dB), .bandWeighted (vector),
% .weights (vector) and .steps.
%
% Example (77.5 dB(A) worked case):
% f = [63 125 250 500 1000 2000 4000 8000];
% L = [70 72 74 76 70.4 71 66 60];
% acoustics.weightedOverall(f, L, 'A').weighted % 77.50
arguments
freqs (1,:) double {mustBePositive}
levels (1,:) double
net (1,1) char {mustBeMember(net,{'A','B','C','Z'})}
end
if numel(freqs) ~= numel(levels)
error('acoustics:weightedOverall:size', 'freqs and levels must match.');
```

Band Workbench (1/3-oct -> octave)

Combine 1/3-octave band SPLs into octave bands, then an overall (and weighted) level.

```
Octave SPL = 10*log10( sum 10^(L_third/10) ) | [over the 3 thirds in that octave]
Overall = 10*log10( sum 10^(L_oct/10) ) | Weighted = 10*log10( sum 10^((L_oct+W_oct)/10) )
```

Source file: +acoustics/bandWorkbench.m

```
function R = bandWorkbench(thirds, levels, net)
%BANDWORKBENCH Combine 1/3-octave levels into octaves and an overall level.
% R = ACOUSTICS.BANDWORKBENCH(thirds, levels, net) takes 1/3-octave centre
% frequencies and their SPLs (dB) and, grouping successive triplets,
% returns octave-band SPLs, the overall SPL and the overall weighted level:
% octave SPL = 10*log10( sum 10^(L_third/10) ) over its 3 thirds
% overall = 10*log10( sum 10^(L_oct/10) )
% weighted = 10*log10( sum 10^((L_oct + W_oct)/10) )
% R has fields .octaveFreq, .octaveSPL (vectors), .overall (dB),
% .weighted (dB) and .steps.
arguments
thirds (1,:) double {mustBePositive}
levels (1,:) double
net (1,1) char {mustBeMember(net,{'A','B','C','Z'})}
end
if numel(thirds) ~= numel(levels)
error('acoustics:bandWorkbench:size', 'thirds and levels must match.');
```

```

    sprintf('(b) Overall SPL      = %.2f dB', R.overall), ...
    sprintf('(b) Overall weighted = %.2f %s', R.weighted, tag)]];
end

```

Time-Averaged Levels (Leq / SEL / Dose)

Leq from levels & durations

Equivalent continuous level (and SEL) from a set of level/duration segments.

```

Leq = 10*log10( (1/T) * sum ti*10^(Li/10) )
SEL = Leq + 10*log10(T / 1 s)
T defaults to sum(durations); pass a fixed T (e.g. 8 h) for LAeq,8h

```

Use case: a sound level meter's readings over several time periods -> LAeq,T and LAeq,8h.

Source file: `+acoustics/leqFromLevels.m`

```

function R = leqFromLevels(levels, durations, opts)
%LEQFROMLEVELS Equivalent continuous level from levels and durations.
% R = ACOUSTICS.LEQFROMLEVELS(levels, durations) with level segments (dB)
% and their durations (seconds) returns:
%   Leq = 10*log10( (1/T) * sum ti*10^(Li/10) )
%   SEL = 10*log10( sum ti*10^(Li/10) / 1 s ) = Leq + 10*log10(T/1s)
% By default T is the sum of the durations; pass 'T',seconds to reference a
% different averaging period (e.g. an 8 h shift or 24 h day). Durations may
% be given in any consistent time unit, but SEL assumes seconds.
% R has fields .Leq (dB), .SEL (dB), .T, .sumT, .energy and .steps.
%
% Example (LAeq,24h = 70.55):
%   acoustics.leqFromLevels([74 69 60], [8 8 8]*3600, 'T', 24*3600).Leq
arguments
    levels      (1,:) double
    durations   (1,:) double {mustBeNonnegative}
    opts.T      double = NaN
end
if numel(levels) ~= numel(durations)
    error('acoustics:leqFromLevels:size', 'levels and durations must match.');
end
R.energy = sum(durations .* 10.^(levels/10));
R.sumT = sum(durations);
if isnan(opts.T) || opts.T <= 0
    R.T = R.sumT;
else
    R.T = opts.T;
end
R.Leq = 10*log10(R.energy/R.T);
R.SEL = 10*log10(R.energy);
R.steps = { ...
    'Leq = 10*log10( (1/T) * sum ti*10^(Li/10) )', ...
    sprintf(' = 10*log10( (1/%.4g) * %.5g ) = %.3f dB', R.T, R.energy, R.Leq), ...
    'SEL = Leq + 10*log10(T/1s)', ...
    sprintf(' = %.3f + 10*log10(%g) = %.2f dB', R.Leq, R.T, R.SEL)};
end

```

Leq from discrete events (pass-bys)

Leq over a reference period from repeated discrete events (e.g. vehicle pass-bys).

```

Leq = 10*log10( (1/T) * sum Ni*ti*10^(Li/10) )
Ni = number of events, ti = single-event duration, T = reference period

```

Source file: `+acoustics/leqFromEvents.m`

```

function R = leqFromEvents(levels, eventTimes, counts, T)
%LEQFROMEVENTS Leq over a period from repeated discrete events (pass-bys).
% R = ACOUSTICS.LEQFROMEVENTS(levels, eventTimes, counts, T) with, per
% event type, its level (dB), single-event duration (s), and number of
% occurrences, over a reference period T (s):
%   Leq = 10*log10( (1/T) * sum Ni*ti*10^(Li/10) )
% R has fields .Leq (dB), .energy, .T and .steps.
arguments
    levels      (1,:) double
    eventTimes  (1,:) double {mustBeNonnegative}
    counts      (1,:) double {mustBeNonnegative}
    T           (1,1) double {mustBePositive}
end
if ~isequal(numel(levels), numel(eventTimes), numel(counts))

```

```

        error('acoustics:leqFromEvents:size', 'Inputs must be the same length.');
```

```

    end
    R.energy = sum(counts .* eventTimes .* 10.^(levels/10));
    R.T = T;
    R.Leq = 10*log10(R.energy/T);
    R.Steps = { ...
        'Leq = 10*log10( (1/T) * sum Ni*ti*10^(Li/10) )', ...
        sprintf(' = 10*log10( (1/%.4g) * %.5g ) = %.3f dB', T, R.energy, R.Leq)};
end
```

Leq time-varying level & percentile LN

Leq and a percentile level LN from a segmented time history, including ramp segments.

Ramp segment: | $L(t) = 10 \cdot \log_{10}(a \cdot t + b) + c$

Leq: integrate $10^{(L(t)/10)}$ dt over all segments, divide by total T, $10 \cdot \log_{10}(\cdot)$

LN: the level exceeded for N% of the total time (e.g. N=10 gives L10)

Use case: a noise event whose level follows a formula (e.g. rising then constant) -> LAeq and LA10.

Note: this tool's logic lives inline inside AcousticsApp.m, not a standalone package function. Below is a clean, tested, standalone rewrite of the same method (**leqTimeVarying.m**) that reproduces the app's results and can be pasted anywhere.

Standalone backup file: leqTimeVarying.m (save with this exact name: can go in ANY folder, no +acoustics package needed)

```

function R = leqTimeVarying(segStrings, N)
%LEQTIMEVARYING Standalone: Leq and percentile LN from level-time segments.
% (Backup equivalent of AcousticsApp's "Leq time-varying level & percentile
% LN" tool -- that tool's ramp integration is built into the app itself
% rather than a package function, so this is a clean standalone rewrite of
% the same method, not a copy of the app's internal code.)
%
% segStrings: cell array of text rows, each either
% 't1,t2,const,L'      level is L dB for t in [t1,t2]
% 't1,t2,ramp,a,b,c'   level is L(t) = 10*log10(a*t+b)+c for t in [t1,t2]
% N: percentile for LN (e.g. 10 for L10); pass [] to skip
%
% R has fields .Leq (dB) and .LN (dB, NaN if N not given).
%
% Example (siren: rises 1st minute per L=10log10(9t+1)+80, then flat 80 for
% 4 more minutes; Leq=82.79 dB(A), L10=87.40 dB(A)):
% R = leqTimeVarying({'0,1,ramp,9,1,80','1,5,const,80'}, 10);
% R.Leq    % 82.79
% R.LN     % 87.40
ns = numel(segStrings);
t1 = zeros(1,ns); t2 = zeros(1,ns); type = strings(1,ns);
a = zeros(1,ns); b = zeros(1,ns); c = zeros(1,ns); L = zeros(1,ns);
for i = 1:ns
    p = strtrim(split(string(segStrings{i}), ','));
    t1(i) = str2double(p(1)); t2(i) = str2double(p(2)); type(i) = lower(p(3));
    if type(i) == "const"
        L(i) = str2double(p(4));
    else
        a(i) = str2double(p(4)); b(i) = str2double(p(5)); c(i) = str2double(p(6));
    end
end
tStart = min(t1); tEnd = max(t2); T = tEnd - tStart;

% --- Leq: exact analytic integral of 10^(L(t)/10) per segment ---
E = 0;
for i = 1:ns
    if type(i) == "const"
        E = E + (t2(i)-t1(i)) * 10^(L(i)/10);
    else
        F = @(t) a(i)*t.^2/2 + b(i)*t;      % integral of (a*t+b) dt
        E = E + (F(t2(i)) - F(t1(i))) * 10^(c(i)/10);
    end
end
R.Leq = 10*log10(E/T);

% --- LN: dense sampling + sort-descending (same convention the app uses) ---
R.LN = NaN;
if nargin >= 2 && ~isempty(N) && N > 0 && N < 100
    np = 200000;
    tt = tStart + ((0:np-1)+0.5)/np*T;
    Lt = zeros(1, np);
    for i = 1:ns
        m = tt >= t1(i) & tt < t2(i);
        if i == ns, m = m | (tt == t2(i)); end    % include the very last instant
        if type(i) == "const"
            Lt(m) = L(i);
        else
            Lt(m) = 10*log10(a(i)*tt + b(i) + c(i));
        end
    end
    R.LN = sort(Lt, 'descend');
end
```

```

        else
            Lt(m) = 10*log10(a(i)*tt(m) + b(i)) + c(i);
        end
    end
    s = sort(Lt, 'descend');
    idx = min(np, max(1, round(N/100*np)));
    R.LN = s(idx);
end
end

```

Noise Dose & max time

Occupational noise dose and LAeq against the OH&S exchange-rate criterion.

Allowed time: | $T_i = T_c / 2^{((L_i - L_c)/q)}$ | [$L_c=85$ dB(A), $q=3$ dB, $T_c=8$ h, defaults]
Dose = $\sum(t_i/T_i)$ | [100% = the limit]
 $LA_{eq,T} = 10 \cdot \log_{10} \left(\frac{1}{T} \sum t_i \cdot 10^{(L_i/10)} \right)$
 $T_{max} = T_c / 2^{((LA_{eq,T} - L_c)/q)}$

Source file: `+acoustics/noiseDose.m`

```

function R = noiseDose(levels, durations, opts)
%NOISEDOSE Occupational noise dose, LAeq and maximum permissible time.
% R = ACOUSTICS.NOISEDOSE(levels, durations) with exposure levels (dB(A))
% and their durations (SECONDS) evaluates a shift against the OH&S
% criterion (defaults: Lc = 85 dB(A), exchange rate q = 3 dB, criterion
% time Tc = 8 h):
%   allowed time Ti = Tc / 2^((Li-Lc)/q)
%   Dose = sum ti/Ti (100% = the limit; ti, Tc in hours)
%   LAeq,T = 10*log10( (1/T) sum ti*10^(Li/10) )
%   LAeq,Tc = 10*log10( (1/Tc) sum ti*10^(Li/10) )
%   Tmax = Tc / 2^((LAeq,T - Lc)/q)
% R has fields .LAeqT, .LAeqTc (dB(A)), .dose (fraction), .dosePct,
% .Tmax (h), .exceeds (logical) and .steps.
arguments
    levels (1,:) double
    durations (1,:) double {mustBeNonnegative}
    opts.Lc (1,1) double = 85
    opts.q (1,1) double {mustBePositive} = 3
    opts.Tc (1,1) double {mustBePositive} = 8
end
if numel(levels) ~= numel(durations)
    error('acoustics:noiseDose:size', 'levels and durations must match.');
```

Max permissible time (steady level)

Maximum daily exposure time for a single steady level, using the OH&S exchange rule.

$T = T_c / 2^{((L - L_c)/q)}$ | [$L_c=85$ dB(A), $q=3$ dB, $T_c=8$ h, defaults]

Use case: 'a worker is exposed to L dB(A); what is the maximum permissible exposure time?'

Source file: `+acoustics/maxPermissibleTime.m`

```

function R = maxPermissibleTime(L, opts)
%MAXPERMISSIBLETIME Maximum daily exposure time for a steady level.
% R = ACOUSTICS.MAXPERMISSIBLETIME(L) for a steady level L (dB(A)) returns
% the permitted exposure time using the OH&S exchange rule (defaults
% Lc = 85 dB(A), q = 3 dB, Tc = 8 h):
%   T = Tc / 2^((L - Lc)/q)
% R has fields .T (h), .exceeds (logical, L > Lc) and .steps.
arguments

```

```

    L (1,1) double
    opts.Lc (1,1) double = 85
    opts.q (1,1) double {mustBePositive} = 3
    opts.Tc (1,1) double {mustBePositive} = 8
end
R.T = opts.Tc/2^((L - opts.Lc)/opts.q);
R.exceeds = L > opts.Lc;
R.steps = { ...
    'T = Tc / 2^((L - Lc)/q)', ...
    sprintf('= %g / 2^((%g - %g)/%g) = %g / %.4g = %.4f h', ...
        opts.Tc, L, opts.Lc, opts.q, opts.Tc, 2^((L-opts.Lc)/opts.q), R.T));
end

```

Stats: SEL <-> Leq

Convert between the Leq over an event and its Sound Exposure Level (1-second reference).

$$\text{SEL} = \text{Leq} + 10 \cdot \log_{10}(T / 1 \text{ s})$$

Source file: `+acoustics/selFromLeq.m`

```

function R = selFromLeq(opts)
%SELFROMLEQ Convert between Leq over an event and its SEL.
% R = ACOUSTICS.SELFROMLEQ('Leq',value,'T',seconds) gives SEL, and
% R = ACOUSTICS.SELFROMLEQ('SEL',value,'T',seconds) gives Leq, using:
% SEL = Leq + 10*log10(T / 1 s)
% Supply the duration T (s) and exactly one of Leq/SEL. R has fields
% .Leq (dB), .SEL (dB), .T and .steps.
arguments
    opts.Leq double = NaN
    opts.SEL double = NaN
    opts.T (1,1) double {mustBePositive} = 1
end
hasLeq = ~isnan(opts.Leq);
hasSEL = ~isnan(opts.SEL);
if hasLeq == hasSEL
    error('acoustics:selFromLeq:input', 'Supply exactly one of Leq or SEL.');
end
R.T = opts.T;
if hasLeq
    R.Leq = opts.Leq;
    R.SEL = R.Leq + 10*log10(R.T);
    R.steps = { ...
        'SEL = Leq + 10*log10(T / 1 s)', ...
        sprintf('= %g + 10*log10(%g) = %g + %.3f = %.2f dB', ...
            R.Leq, R.T, R.Leq, 10*log10(R.T), R.SEL)};
else
    R.SEL = opts.SEL;
    R.Leq = R.SEL - 10*log10(R.T);
    R.steps = { ...
        'Leq = SEL - 10*log10(T / 1 s)', ...
        sprintf('= %g - 10*log10(%g) = %.2f dB', R.SEL, R.T, R.Leq)};
end
end

```

Hearing Protection

Hearing protector (SLC80)

Protected A-weighted level using the single-number SLC80 (AS/NZS 1269) method.

$$\text{LAeq}' = \text{LCeq} - \text{SLC80}$$

(SLC80 subtracts from the C-weighted level, not the A-weighted one)

Source file: `+acoustics/hearingProtector.m`

```

function R = hearingProtector(LCeq, SLC80)
%HEARINGPROTECTOR Protected A-weighted level using the SLC80 method.
% R = ACOUSTICS.HEARINGPROTECTOR(LCeq, SLC80) estimates the A-weighted level
% under a hearing protector (ear muff/plug) using the AS/NZS 1269 Sound Level
% Conversion method:
% LAeq' = LCeq - SLC80
% where LCeq is the C-weighted equivalent noise level (dB(C)) and SLC80 is
% the protector's Sound Level Conversion rating (dB). Note the SLC80 is
% subtracted from the C-weighted level, not the A-weighted one.
%
% (Tip: LCeq comes from C-weighting the octave bands and energy-averaging
% over time. When the C-weighting is ~0 dB in the important bands, LCeq
% equals the linear Leq.)

```



```

%
% R has fields .protectedLAeq (dB(A)) and .steps.
%
% Example (Lcq,8h = Lcq = 99.72 dB, muffs SLC80 = 27 dB):
% acoustics.hearingProtector(99.721, 27).protectedLAeq % 72.72 dB(A)
arguments
    Lcq (1,1) double
    SLC80 (1,1) double {mustBeNonnegative}
end
R.protectedLAeq = Lcq - SLC80;
R.steps = { ...
    'Protected level (AS/NZS 1269, SLC80 method):', ...
    'LAeq' = Lcq - SLC80 (SLC80 subtracted from the C-weighted level)', ...
    sprintf(' = %.2f - %.2f = %.2f dB(A)', Lcq, SLC80, R.protectedLAeq));
end

```

Hearing protector (octave-band mean/SD)

A-weighted level at the ear from octave-band mean + standard-deviation attenuation data.

Assumed attenuation per band = mean - k*SD | [k=1 'maximum'/worst-case, Bies & Hansen; k=0 mean; k=2 mean-2SD]

$L_{ear} = L_p - (mean - k*SD)$

$L_{ear,A} = L_{ear} + A\text{-weighting}(freq) \quad | \quad LA = 10*\log_{10}(\sum 10^{(L_{ear,A}/10)})$

Use case: NAL octave-band muff/plug attenuation data + environment octave SPLs -> max LA at the ear.

Source file: `+acoustics/hearingProtectorBands.m`

```

function R = hearingProtectorBands(freq, Lp, meanAtt, stdAtt, opts)
%HEARINGPROTECTORBANDS A-weighted level at the ear, octave-band mean+/-SD method.
% R = ACOUSTICS.HEARINGPROTECTORBANDS(freq, Lp, meanAtt, stdAtt) estimates
% the A-weighted sound pressure level under a hearing protector from octave
% band data (all inputs same-length vectors):
%   freq octave band centre frequencies (Hz)
%   Lp octave band sound pressure levels (dB)
%   meanAtt mean protector attenuation per band (dB)
%   stdAtt standard deviation of the attenuation per band (dB)
% Per band the assumed attenuation is meanAtt - k*stdAtt (least/worst-case
% protection for k>0, i.e. the "maximum" ear level), so
%   L_ear = Lp - (meanAtt - k*stdAtt)
%   L_ear,A = L_ear + A-weighting(freq)
%   LA = 10*log10( sum 10^(L_ear,A/10) )
% Option 'k' sets the SD multiplier (default 1): k=0 mean only, k=1
% (mean-1SD, Bies & Hansen), k=2 (mean-2SD). 'net' picks the weighting
% network ('A' default, 'Z' for unweighted).
%
% R has fields .LA (protected, weighted), .LAunprot (no protector, same
% network), .reduction (dB), .perBand (a table: freq Lp assumedAtt L_ear
% L_ear_weighted) and .steps.
%
% Example (NAL EML-45 muffs, k=1 -> LA = 93.0 dB(A)):
% f=[125 250 500 1000 2000 4000 8000];
% Lp=[102 103 104 95 91 98 98];
% m=[5.3 8.5 15.9 26 25.4 35.7 27.2];
% s=[4.6 2.4 4.6 4.3 5.5 4.1 5.5];
% acoustics.hearingProtectorBands(f,Lp,m,s).LA
arguments
    freq (1,:) double {mustBePositive}
    Lp (1,:) double
    meanAtt (1,:) double
    stdAtt (1,:) double
    opts.k (1,1) double = 1
    opts.net (1,1) char {mustBeMember(opts.net,{'A','B','C','Z'})} = 'A'
end
n = numel(freq);
if ~all([numel(Lp) numel(meanAtt) numel(stdAtt)] == n)
    error('acoustics:hearingProtectorBands:size', ...
        'freq, Lp, meanAtt and stdAtt must be the same length.');
```

```

if opts.k == 0, kstr = 'mean'; end
R.steps = { ...
    sprintf('Assumed attenuation per band = %s', kstr), ...
    sprintf('L_ear = Lp - (mean - %g*SD), then %s-weight and energy-sum', opts.k, opts.net), ...
    sprintf('L%c (under protector) = %.2f dB', opts.net, R.LA), ...
    sprintf('L%c (no protector) = %.2f dB -> reduction %.2f dB', ...
        opts.net, R.LAunprot, R.reduction));
end

```

Loudness

Loudness: phons -> sones

Convert loudness level (phons, from equal-loudness contours) to loudness (sones).

$$S = 2^{((LL - 40)/10)} \quad | \quad [\text{valid for } LL \geq 40 \text{ phon}]$$

Source file: `+acoustics/phonToSone.m`

```

function R = phonToSone(phon)
%PHONTOSONE Convert loudness level (phons) to loudness (sones).
% R = ACOUSTICS.PHONTOSONE(phon) returns
% S = 2^((LL - 40)/10)
% valid for LL >= 40 phon. R has fields .sones and .steps.
%
% Example:
% acoustics.phonToSone(80).sones    % 16 sones
arguments
    phon (1,1) double
end
R.sones = 2^((phon - 40)/10);
R.steps = { ...
    'S = 2^((LL - 40)/10)', ...
    sprintf(' = 2^((%g - 40)/10) = %.3f sones', phon, R.sones));
if phon < 40
    R.steps{end+1} = 'Note: formula assumes LL >= 40 phon.';
end
end

```

Loudness: sones -> phons

Convert loudness (sones) back to loudness level (phons).

$$LL = 40 + 10 \cdot \log_2(S)$$

Source file: `+acoustics/soneToPhon.m`

```

function R = soneToPhon(sones)
%SONETOPHON Convert loudness (sones) to loudness level (phons).
% R = ACOUSTICS.SONETOPHON(sones) returns
% LL = 40 + 10*log2(S)
% R has fields .phons and .steps.
%
% Example:
% acoustics.soneToPhon(16).phons    % 80 phon
arguments
    sones (1,1) double {mustBePositive}
end
R.phons = 40 + 10*log2(sones);
R.steps = { ...
    'LL = 40 + 10*log2(S)', ...
    sprintf(' = 40 + 10*log2(%g) = %.2f phons', sones, R.phons));
end

```

Speech & Community Noise

Speech: PSIL & voice effort

Speech interference level from octave-band noise, and the resulting required voice effort.

SIL = mean(octave band levels)

Classic SIL: 500/1000/2000/4000 Hz. | Preferred PSIL: 500/1000/2000 Hz

Compare SIL (or the resulting voice level) against distance to judge communication effort

Source file: `+acoustics/speechInterferenceLevel.m`

```

function R = speechInterferenceLevel(bandLevels)
%SPEECHINTERFERENCELEVEL Speech interference level (SIL / PSIL).
% R = ACOUSTICS.SPEECHINTERFERENCELEVEL(bandLevels) returns the arithmetic
% mean of the octave-band noise levels (dB) that interfere with speech:
%     SIL = mean( L_bands )
% Classic SIL uses the 500, 1000, 2000 and 4000 Hz octave bands; the
% preferred-octave PSIL uses 500, 1000 and 2000 Hz. Pass whichever set the
% question specifies. R has fields .SIL (dB) and .steps.
%
% Example (ship engine-room):
%     acoustics.speechInterferenceLevel([105 104 103 102.68]).SIL    % 103.67
arguments
    bandLevels (1,:) double {mustBeNonempty}
end
R.SIL = mean(bandLevels);
terms = strjoin(arrayfun(@(x) sprintf('%g', x), bandLevels, ...
    'UniformOutput', false), ' + ');
R.steps = { ...
    'SIL = mean( octave-band levels )', ...
    sprintf(' = (%s)/%d = %.2f dB', terms, numel(bandLevels), R.SIL)};
end

```

Community: day-night level Ldn

Community day-night average level, applying the standard 10 dB night-time penalty.

$$L_{dn} = 10 \cdot \log_{10} \left(\frac{1}{24} \left[15 \cdot 10^{(L_{day}/10)} + 9 \cdot 10^{((L_{night}+10)/10)} \right] \right)$$
 [15 h day, 9 h night, +10 dB night penalty]

Source file: `+acoustics/dayNightLevel.m`

```

function R = dayNightLevel(Lday, Lnight)
%DAYNIGHTLEVEL Community day-night average sound level Ldn.
% R = ACOUSTICS.DAYNIGHTLEVEL(Lday, Lnight) with the 15 h daytime LAeq and
% the 9 h night-time LAeq (dB(A)) applies the 10 dB night penalty:
%     Ldn = 10*log10( (1/24)[ 15*10^(Lday/10) + 9*10^((Lnight+10)/10) ] )
% R has fields .Ldn (dB(A)) and .steps.
arguments
    Lday (1,1) double
    Lnight (1,1) double
end
ed = 15*10^(Lday/10);
en = 9*10^((Lnight+10)/10);
R.Ldn = 10*log10((ed + en)/24);
R.steps = { ...
    'Ldn = 10*log10( (1/24)[ 15*10^(Lday/10) + 9*10^((Lnight+10)/10) ] )', ...
    sprintf(' = 10*log10( (1/24)[ %.4g + %.4g ] )', ed, en), ...
    sprintf(' = 10*log10( %.5g ) = %.2f dB(A)', (ed+en)/24, R.Ldn)};
end

```

Statistics

Percentile level LN (part of the time-varying Leq tool)

Statistical level LN exceeded for N% of a sampled time history.

LN = the level exceeded for N% of the time history
 e.g. N=10 -> L10, | N=90 -> L90

Source file: `+acoustics/percentileLevel.m`

```

function R = percentileLevel(samples, N)
%PERCENTILELEVEL Statistical level LN exceeded N% of the time.
% R = ACOUSTICS.PERCENTILELEVEL(samples, N) returns LN, the level exceeded
% for N% of the (equally sampled) time history in SAMPLES (dB). For
% example N=10 gives L10 and N=90 gives L90.
% R has fields .LN (dB) and .steps.
arguments
    samples (1,:) double {mustBeNonempty}
    N (1,1) double
end
if ~(N > 0 && N < 100)
    error('acoustics:percentileLevel:N', 'N must be between 0 and 100.');
```

```

    sprintf('Sort %d samples descending; take the value at the N=%g%% rank.', n, N), ...
    sprintf('L%g = %.2f dB (exceeded %g%% of the time)', N, R.LN, N));
end

```

Stats: sort values into terms

Assign a set of unlabelled dB values to L1 / Leq / L99 / SEL by their expected relative magnitude.

Sort the given values in DESCENDING order, then assign by physical meaning:

```

SEL | (largest - normalised to 1 s, energy-concentrated)
L1  | (2nd largest - level exceeded only 1% of the time, near the peak)
Leq | (3rd - the time/energy average, between the extremes)
L99 | (smallest - level exceeded 99% of the time, i.e. the background)

```

Use case: 'the values X, Y, Z, W were measured for L1, L99, Leq and SEL but not which is which.'

Note: this tool's logic lives inline inside AcousticsApp.m, not a standalone package function. Below is a clean, tested, standalone rewrite of the same method (**sortValuesIntoTerms.m**) that reproduces the app's results and can be pasted anywhere.

Standalone backup file: sortValuesIntoTerms.m (save with this exact name: can go in ANY folder, no +acoustics package needed)

```

function R = sortValuesIntoTerms(values)
%SORTVALUESINTOTERMS Standalone: assign four dB values to SEL/L1/Leq/L99.
% (Backup equivalent of AcousticsApp's "Stats: sort values into terms"
% tool -- that tool's logic is a short inline heuristic in the app itself
% rather than a package function, so this is the same method as a
% standalone rewrite.)
%
% R = SORTVALUESINTOTERMS(values) takes the four unlabelled dB values and
% assigns them by DESCENDING magnitude to the terms that are physically
% largest to smallest:
%   SEL (largest) - 1-second-normalised, energy-concentrated
%   L1  (2nd)    - level exceeded only 1% of the time, near the peak
%   Leq (3rd)    - the time/energy average, between the extremes
%   L99 (smallest) - level exceeded 99% of the time, i.e. background
%
% R has fields .SEL, .L1, .Leq, .L99 (dB).
%
% Example:
%   R = sortValuesIntoTerms([102.5 31.5 84.4 93.5]);
%   R.SEL % 102.5  R.L1 % 93.5  R.Leq % 84.4  R.L99 % 31.5
%   if numel(values) ~= 4
%       error('sortValuesIntoTerms:size', 'Enter exactly four values.');
```

Sound Insulation (Transmission Loss)

Insulation: mass law TL

Normal-incidence transmission loss of a limp panel from the mass law; layers in contact ADD mass.

$TL = 20 \cdot \log_{10}(M \cdot f) - 42.4$ | [M = surface mass, kg/m²; layers/glued panels: M = sum of layer masses]

Use case: 'find the mass-law TL of a Xkg/m² panel at several frequencies.'

Source file: +acoustics/massLawTL.m

```

function R = massLawTL(f, opts)
%MASSLAWTL Transmission loss of a limp panel from the mass law.
% R = ACOUSTICS.MASSLAWTL(f, 'M', m) at frequency f (Hz) for a panel of
% surface mass m (kg/m^2):
%   TL = 20*log10(M*f) - 42.4
%
% Layered / glued / laminated panels (bonded together with NO air gap act
% as one leaf, so their surface masses ADD): pass M as a vector, e.g.
%   acoustics.massLawTL(100, 'M', [10 10]) % two 10 kg/m^2 leaves -> M=20
% Likewise 'density' and 'thickness_mm' may be vectors (one entry per
% layer); the layer masses density.*thickness/1000 are summed.
%
% 'constant' overrides the 42.4 dB term (normal incidence). R has fields

```

```
% .M (total kg/m^2), .layers (per-layer kg/m^2), .TL (dB) and .steps.
%
% Example (plywood 3 mm at 500 kg/m^3, 1 kHz):
%   acoustics.massLawTL(1000,'density',500,'thickness_mm',3).TL % ~21 dB
arguments
    f (1,1) double {mustBePositive}
    opts.M          double = NaN
    opts.density     double = NaN
    opts.thickness_mm double = NaN
    opts.constant (1,1) double = 42.4
end
if ~any(isnan(opts.M))
    layers = opts.M(:).';
elseif ~any(isnan(opts.density)) && ~any(isnan(opts.thickness_mm))
    d = opts.density(:).'; th = opts.thickness_mm(:).';
    if numel(d) ~= numel(th)
        error('acoustics:massLawTL:layers', ...
            'density and thickness_mm must have the same number of layers.');
```

```

if ~isempty(opts.TL) % Tls supplied directly
    tl = opts.TL(:).';
    R.f = NaN;
    R.TLpanels = tl;
    R.TL = sum(tl);
    R.alphaT = prod(10.^(-tl/10));
    R.steps = { ...
        sprintf('Panels in series: alpha_t = prod(alpha_i), TL = sum(TL_i)'), ...
        sprintf('TL = %s = %.1f dB', ...
            strjoin(arrayfun(@(x) sprintf('%.2f',x), tl, 'UniformOutput',false), ' + '), R.TL), ...
        sprintf('alpha_t = %.4g', R.alphaT)};
    return;
end

if isempty(opts.masses) || any(isnan(f))
    error('acoustics:doublePanelTL:input', ...
        'Provide masses (and frequency f), or TL values directly.');
```

```

end
m = opts.masses(:).';
f = f(:).';
R.f = f;
R.TLpanels = zeros(numel(f), numel(m));
R.TL = zeros(size(f));
R.alphaT = zeros(size(f));
lines = {'Panels in series: alpha_t = prod(alpha_i) -> TL = sum(TL_i)', ...
    'Each panel (mass law): TLi = 20*log10(mi*f) - 42.4', ''};
for i = 1:numel(f)
    tli = 20*log10(m*f(i)) - K;
    R.TLpanels(i,:) = tli;
    R.TL(i) = sum(tli);
    R.alphaT(i) = prod(10.^(-tli/10));
    lines{end+1} = sprintf('f = %g Hz: TL = %s = %.1f dB', f(i), ...
        strjoin(arrayfun(@(x) sprintf('%.2f',x), tli, 'UniformOutput',false), ' + '), ...
        R.TL(i)); %#ok<AGROW>
end
R.steps = lines;
end

```

Insulation: interface impedance & coeffs

Reflection/transmission at a normal-incidence interface between two media (e.g. air-water).

```

r = z2/z1 | alpha_t = 4r/(r+1)^2 | (power transmitted) | alpha_r = ((r-1)/(r+1))^2
(reflected)
Tp = 2*z2/(z1+z2) | (pressure transmission coeff) | Rp = (z2-z1)/(z1+z2) | (pressure
reflection coeff)
TL = -10*log10(alpha_t)
pTrans = Tp*p0 | pInterface = pTrans | uTrans = pTrans/z2 | pRefl = Rp*p0

```

Use case: plane wave normally incident on an interface (e.g. air-water); find transmitted p, u, and tau.

Source file: `acoustics/interfaceImpedance.m`

```

function R = interfaceImpedance(z1, z2, opts)
%INTERFACEIMPEDANCE Transmission/reflection at a normal-incidence interface.
% R = ACOUSTICS.INTERFACEIMPEDANCE(z1, z2) for characteristic impedances
% z1 (incident medium) and z2 (second medium) in rayls returns:
%
%   r      = z2/z1
%   alpha_t = 4r / (r+1)^2      (transmitted intensity/power fraction)
%   alpha_r = ((r-1)/(r+1))^2   (reflected intensity fraction)
%   TL      = -10*log10(alpha_t)
%   Tp      = 2*z2/(z1+z2)      (pressure amplitude transmission coeff)
%   Rp      = (z2-z1)/(z1+z2)   (pressure amplitude reflection coeff)
%
% Give the incident wave amplitude EITHER as a peak pressure ('p0', Pa) OR
% as a sound pressure level ('Lp', dB re 2e-5 Pa); with 'Lp' the incident
% PEAK amplitude is p0 = sqrt(2)*2e-5*10^(Lp/20). Then:
%   pTrans  = Tp * p0           transmitted pressure amplitude (Pa)
%   pInterface = pTrans         total pressure at the interface (Pa)
%                               (= incident+reflected, by continuity)
%   uTrans  = pTrans / z2       transmitted particle velocity amp (m/s)
%   pRefl   = Rp * p0          reflected pressure amplitude (Pa)
%
% R has fields .ratio, .alphaT, .alphaR, .TL, .Tp, .Rp and (if p0/Lp given)
% .pIncident, .pInterface, .pTrans, .uTrans, .pRefl, plus .steps.
%
% Example (107 dB, 1000 Hz, air onto water; z1=415, z2=1.5e6):
%   R = acoustics.interfaceImpedance(1.21*343, 1000*1500, 'Lp', 107);
%   R.pIncident % 6.33 Pa (peak)   R.pInterface % 12.66 Pa

```

```

%      R.uTrans      % 8.44e-6 m/s      R.alphaT      % 0.00111
arguments
    z1 (1,1) double {mustBePositive}
    z2 (1,1) double {mustBePositive}
    opts.p0 (1,1) double = NaN      % NaN default: no validator (MATLAB validates defaults)
    opts.Lp (1,1) double = NaN
end
R.ratio = z2/z1;
R.alphaT = 4*R.ratio/((R.ratio+1)^2);
R.alphaR = ((R.ratio-1)/(R.ratio+1))^2;
R.TL = -10*log10(R.alphaT);
R.Tp = 2*z2/(z1+z2);
R.Rp = (z2-z1)/(z1+z2);
R.steps = { ...
    sprintf('r = z2/z1 = %.6g / %.6g = %.6g', z2, z1, R.ratio), ...
    sprintf('Tp = 2 z2/(z1+z2) = %.6g (pressure transmission coeff)', R.Tp), ...
    sprintf('Rp = (z2-z1)/(z1+z2) = %.6g (pressure reflection coeff)', R.Rp), ...
    sprintf('alpha_t = 4r/(r+1)^2 = %.6g (sound/power transmission coeff)', R.alphaT), ...
    sprintf('alpha_r = ((r-1)/(r+1))^2 = %.6g', R.alphaR), ...
    sprintf('TL = -10*log10(alpha_t) = %.2f dB', R.TL)};
p0 = opts.p0;
if ~isnan(opts.Lp)
    prms = 2e-5 * 10^(opts.Lp/20);
    p0 = sqrt(2) * prms;          % incident PEAK amplitude
    R.steps = [R.steps, { ...
        sprintf('p_rms = 2e-5*10^(Lp/20) = 2e-5*10^(%.4g/20) = %.4g Pa', opts.Lp, prms), ...
        sprintf('(a) incident peak p0 = sqrt(2)*p_rms = %.4g Pa', p0)}];
end
if ~isnan(p0)
    R.pIncident = p0;
    R.pTrans = R.Tp * p0;
    R.pInterface = R.pTrans;      % total at interface = transmitted
    R.uTrans = R.pTrans / z2;
    R.pRefl = R.Rp * p0;
    R.steps = [R.steps, { ...
        sprintf('(b) total pressure at interface = Tp*p0 = %.6g Pa', R.pInterface), ...
        sprintf('(c) transmitted pressure in medium 2 = %.6g Pa (= b, continuity)', R.pTrans), ...
        sprintf('(d) uTrans = pTrans/z2 = %.6g / %.6g = %.4g m/s', R.pTrans, z2, R.uTrans), ...
        sprintf('(e) sound transmission coeff alpha_t = %.4g', R.alphaT), ...
        sprintf('reflected pressure pRefl = Rp*p0 = %.6g Pa', R.pRefl)}];
end
end
end

```

Insulation: TL from coefficient

Transmission loss directly from a known transmission coefficient.

$TL = -10 \cdot \log_{10}(\alpha_t)$

Source file: `+acoustics/tlFromCoefficient.m`

```

function R = tlFromCoefficient(alphaT)
%TLFROMCOEFFICIENT Transmission loss from a transmission coefficient.
% R = ACOUSTICS.TLFROMCOEFFICIENT(alphaT) returns
% TL = -10*log10(alpha_t)
% for 0 < alpha_t <= 1. R has fields .TL (dB) and .steps.
arguments
    alphaT (1,1) double
end
if ~(alphaT > 0 && alphaT <= 1)
    error('acoustics:tlFromCoefficient:range', 'alpha_t must be in (0, 1].');
end
R.TL = -10*log10(alphaT);
R.steps = { ...
    'TL = -10*log10(alpha_t)', ...
    sprintf(' = -10*log10(%.6g) = %.2f dB', alphaT, R.TL)};
end

```

Insulation: panel resonance frequency

Fundamental resonance frequency of a mass-spring (limp) panel.

$f_n = (1/2 \cdot \pi) \cdot \sqrt{K / M}$ | [K = stiffness/area N/m^3 , M = surface mass kg/m^2]

Source file: `+acoustics/panelResonance.m`

```

function R = panelResonance(K, M)
%PANELRESONANCE Fundamental resonance frequency of a mass-spring panel.
% R = ACOUSTICS.PANELRESONANCE(K, M) with stiffness per unit area
% K (N/m^3) and surface mass M (kg/m^2) returns:
% fn = (1/2pi) * sqrt(K/M)
% R has fields .fn (Hz) and .steps.

```

```

arguments
    K (1,1) double {mustBePositive}
    M (1,1) double {mustBePositive}
end
R.fn = sqrt(K/M)/(2*pi);
R.steps = { ...
    'fn = (1/2pi)*sqrt(K/M)', ...
    sprintf(' = (1/2pi)*sqrt(%g/%g) = %.2f Hz', K, M, R.fn));
end

```

Insulation: panel TL (resonant, mass+stiffness)

TL of a resonant panel (mass-spring model) across stiffness- and mass-controlled regions.

$k = F/\text{deflection}$ (from a static test) | $s'' = k/A$ | $m' = \text{density} \times \text{thickness}$
 $fn = (1/2\pi) \cdot \sqrt{s''/m'}$
 $f < fn$ (stiffness-controlled): | $TL = 20 \cdot \log_{10}(s'' / (2 \cdot \rho \cdot c \cdot w))$)
 $f \geq fn$ (mass-controlled): | $TL = 20 \cdot \log_{10}(w \cdot m' / (2 \cdot \rho \cdot c))$), | $w = 2 \cdot \pi \cdot f$

Use case: a panel deflects a known amount under a static force -> find fn and TL above/below resonance.

Source file: `acoustics/panelTL.m`

```

function R = panelTL(freqs, F, deflection, L, W, thickness, density, opts)
%PANELTL Transmission loss of a resonant single panel (mass + stiffness).
% R = ACOUSTICS.PANELTL(freqs, F, deflection, L, W, thickness, density)
% models a panel as an undamped mass-spring system and returns its natural
% frequency and the sound transmission loss at each frequency in FREQS (Hz).
%
% The stiffness comes from a static test (force F in N deflecting the centre
% by DEFLECTION in m); the mass from the panel geometry and density:
%
%   A   = L*W                panel area (m^2)
%   k   = F / deflection      total stiffness (N/m)
%   m'  = density * thickness surface mass (kg/m^2)
%   s'' = k / A              stiffness per unit area (N/m^3)
%   fn  = (1/2pi) sqrt(s''/m') natural frequency (Hz)
% The TL uses the DOMINANT reactance in each region (the course method):
%   f < fn (stiffness-controlled): TL = 20*log10( s'' / (2 rho c * w) )
%   f >= fn (mass-controlled/mass law): TL = 20*log10( w*m' / (2 rho c) )
% with w = 2*pi*f. All lengths in metres. 'rhoc' defaults to 415 rayls.
%
% R has fields .area, .k, .surfaceMass, .stiffnessPerArea, .fn, .TL (1xN),
% .region (1xN string), .freqs and .steps.
%
% Example (skylight glass panel, 197x98 mm, 14 mm, 1045 N -> 0.8 mm):
%   R = acoustics.panelTL([100 1000], 1045, 0.8e-3, 0.197, 0.098, 0.014, 2500);
%   R.fn    % 221.3 Hz
%   R.TL    % [42.3 48.5] dB
arguments
    freqs      (1,:) double {mustBePositive}
    F          (1,1) double {mustBePositive}
    deflection (1,1) double {mustBePositive}
    L          (1,1) double {mustBePositive}
    W          (1,1) double {mustBePositive}
    thickness  (1,1) double {mustBePositive}
    density    (1,1) double {mustBePositive}
    opts.rhoc  (1,1) double {mustBePositive} = 415
end
R.area = L*W;
R.k = F/deflection;
R.surfaceMass = density*thickness;
R.stiffnessPerArea = R.k/R.area;
R.fn = sqrt(R.stiffnessPerArea/R.surfaceMass)/(2*pi);
R.freqs = freqs;

w = 2*pi*freqs;
Xmass = w*R.surfaceMass;           % mass reactance
Xstiff = R.stiffnessPerArea./w;    % stiffness reactance
below = freqs < R.fn;
X = Xmass; X(below) = Xstiff(below); % dominant reactance per region
R.TL = 20*log10(X/(2*opts.rhoc));
R.region = repmat("mass-controlled (mass law)", 1, numel(freqs));
R.region(below) = "stiffness-controlled";

lines = { ...
    'RESONANT SINGLE PANEL (mass + stiffness, no damping)', ...
    sprintf('A = L*W = %.5g m^2 . m' = rho*t = %.1f kg/m^2', R.area, R.surfaceMass), ...
    sprintf('k = F/deflection = %g/%g = %.4g N/m . s' = k/A = %.4g N/m^3', ...
        F, deflection, R.k, R.stiffnessPerArea), ...
    sprintf('(a) fn = (1/2pi) sqrt(s''/m') = %.1f Hz', R.fn), ...

```



```

    'Below fn: TL = 20*log10(s"/(2 rho c w)); above fn: TL = 20*log10(w m"/(2 rho c))');
for i = 1:numel(freqs)
    lines(end+1) = sprintf('    f = %g Hz [%s] -> TL = %.1f dB', ...
        freqs(i), R.region(i), R.TL(i)); %#ok<AGROW>
end
R.steps = lines;
end

```

Insulation: barrier added to a spectrum (dB(A) before/after)

Effect of inserting a mass-law barrier into an octave-band spectrum: overall dB(A) before/after.

Per band: $TL = 20 \cdot \log_{10}(m' \cdot f) - 42.4$ | $Lp_after = Lp - TL$
 Overall (weighted) = $10 \cdot \log_{10}(\sum 10^{((Lp + Wi)/10)})$ | [before and after]

Use case: 'a barrier/disc is fitted in a duct/tunnel; find the overall dB(A) before and after.'

Source file: `+acoustics/barrierInsertion.m`

```

function R = barrierInsertion(freqs, Lp, opts)
%BARRIERINSERTION Effect of a mass-law barrier on an octave-band spectrum.
% R = ACOUSTICS.BARRIERINSERTION(freqs, Lp, 'density', d, 'thickness_mm', t)
% takes octave-band sound pressure levels Lp (dB) and a limp barrier (e.g. a
% fitted panel or disc) and returns the A-weighted overall level before and
% after the barrier is inserted, plus the reduction.
%
% Per band the barrier adds its mass-law transmission loss:
% m' = density*thickness (or pass 'M', surface mass kg/m^2)
% TL = 20*log10(m'*f) - 42.4
% Lp_after = Lp - TL
% The overall weighted levels use the standard A/B/C table (default 'net','A'):
% (i) overall before = 10*log10( sum 10^((Lp + Wi)/10) )
% (ii) overall after = 10*log10( sum 10^((Lp_after + Wi)/10) )
%
% R has fields .TL, .LpAfter (1xB), .before, .after (dB), .reduction, .steps.
%
% Example (tunnel + 25 mm foam disc, rho 320 kg/m^3):
% R = acoustics.barrierInsertion([250 500 1000 2000], [104 109 106 110], ...
%     'density', 320, 'thickness_mm', 25);
% R.before % 113.3 dB(A)
% R.after % 78.8 dB(A)
arguments
    freqs (1,:) double {mustBePositive}
    Lp (1,:) double
    opts.M double = NaN
    opts.density double = NaN
    opts.thickness_mm double = NaN
    opts.net (1,1) char {mustBeMember(opts.net,{'A','B','C','Z'})} = 'A'
end
if numel(freqs) ~= numel(Lp)
    error('acoustics:barrierInsertion:size', 'freqs and Lp must match.');
```

Mufflers / Silencers

Muffler: sudden area change

Transmission loss of a sudden pipe cross-section change (reactive muffler element).

$$Tt = 4*S1*S2 / (S1+S2)^2 \quad | \quad TL = -10*\log_{10}(Tt)$$

Source file: `+acoustics/mufflerAreaChange.m`

```
function R = mufflerAreaChange(S1, S2)
%MUFLERAREACHANGE Transmission loss of a sudden cross-section change.
% R = ACOUSTICS.MUFLERAREACHANGE(S1, S2) for a pipe of area S1 (m^2)
% opening into an area S2 (m^2):
%   Tt = 4*S1*S2 / (S1+S2)^2      (transmitted power fraction)
%   TL = -10*log10(Tt)
% R has fields .Tt, .TL (dB) and .steps.
arguments
    S1 (1,1) double {mustBePositive}
    S2 (1,1) double {mustBePositive}
end
R.Tt = 4*S1*S2/((S1+S2)^2);
R.TL = -10*log10(R.Tt);
R.steps = { ...
    'Tt = 4*S1*S2 / (S1+S2)^2', ...
    sprintf(' = %.4g / %.4g = %.4g', 4*S1*S2, (S1+S2)^2, R.Tt), ...
    sprintf('TL = -10*log10(Tt) = %.2f dB', R.TL)};
end
```

Muffler: simple expansion chamber

TL of a simple (reactive) expansion-chamber muffler at a given frequency.

$$m = S2/S1 \quad | \quad kL = (2*\pi*f/c) * L$$
$$TL = 10*\log_{10}[\cos^2(kL) + 1/4 (m + 1/m)^2 \sin^2(kL)]$$

Source file: `+acoustics/expansionChamberTL.m`

```
function R = expansionChamberTL(S1, S2, L, f, opts)
%EXPANSIONCHAMBERTL Transmission loss of a simple expansion chamber.
% R = ACOUSTICS.EXPANSIONCHAMBERTL(S1, S2, L, f) for a chamber of area
% S2 (m^2) and length L (m) between pipes of area S1 (m^2) at frequency
% f (Hz):
%   m = S2/S1,    kL = 2*pi*f/c * L
%   TL = 10*log10[ cos^2(kL) + 1/4 (m + 1/m)^2 sin^2(kL) ]
% Optional 'c' (default 343 m/s). R has fields .m, .kL (rad), .TL (dB),
% .lambda (m) and .steps.
arguments
    S1 (1,1) double {mustBePositive}
    S2 (1,1) double {mustBePositive}
    L (1,1) double {mustBePositive}
    f (1,1) double {mustBePositive}
    opts.c (1,1) double {mustBePositive} = 343
end
R.m = S2/S1;
R.kL = 2*pi*f/opts.c*L;
R.TL = 10*log10(cos(R.kL)^2 + 0.25*(R.m + 1/R.m)^2*sin(R.kL)^2);
R.lambda = opts.c/f;
R.steps = { ...
    sprintf('m = S2/S1 = %.2f', R.m), ...
    sprintf('kL = 2*pi*f/c*L = %.3f rad', R.kL), ...
    'TL = 10*log10[ cos^2(kL) + 1/4 (m+1/m)^2 sin^2(kL) ]', ...
    sprintf(' = %.2f dB    (lambda = %.3f m, lambda/4 = %.3f m)', R.TL, R.lambda, R.lambda/4)};
end
```

Muffler: TL / IL / NR (level difference)

The common level-difference definition underlying TL, insertion loss and noise reduction.

$$TL = L_{w1} - L_{w2} \quad | \quad (\text{transmission loss}) \quad | \quad IL = L_{\text{before}} - L_{\text{after}} \quad | \quad (\text{insertion loss})$$
$$NR = L_{\text{in}} - L_{\text{out}} \quad | \quad (\text{noise reduction})$$

Source file: `+acoustics/levelDifference.m`

```
function R = levelDifference(before, after)
%LEVELDIFFERENCE Level difference for TL / IL / NR.
% R = ACOUSTICS.LEVELDIFFERENCE(before, after) returns the difference
% between two levels (dB). The same subtraction underlies:
%   TL = Lw1 - Lw2      (transmission loss)
%   IL = L_before - L_after (insertion loss)
```

```

%      NR = L_in - L_out      (noise reduction)
%      R has fields .difference (dB) and .steps.
arguments
    before (1,1) double
    after  (1,1) double
end
R.difference = before - after;
R.steps = { ...
    sprintf('Difference = %g - %g = %.2f dB', before, after, R.difference), ...
    '(TL = Lw1 - Lw2 · IL = L_before - L_after · NR = L_in - L_out)'};
end

```

Support Files

These three files are shared dependencies -- many calculators above call them internally (acoustics.constants(), acoustics.weightingValue(), acoustics.weightingTable()). Always include all three in your +acoustics folder, regardless of which calculators you need.

constants.m

Source file: +acoustics/constants.m

```
function C = constants()
%CONSTANTS Course physical constants for MECH9325 acoustics.
% C = ACOUSTICS.CONSTANTS() returns a struct of the reference quantities
% used throughout the toolkit:
%     PREF reference sound pressure      2e-5 Pa
%     WREF reference sound power        1e-12 W
%     IREF reference sound intensity    1e-12 W/m^2
%     RHOC characteristic impedance of air 415 rayls
%     CAIR speed of sound in air at 20 C 343 m/s
%
% Example:
%     C = acoustics.constants;   C.RHOC    % -> 415
C = struct( ...
    'PREF', 2e-5, ...
    'WREF', 1e-12, ...
    'IREF', 1e-12, ...
    'RHOC', 415, ...
    'CAIR', 343);
end
```

weightingTable.m

Source file: +acoustics/weightingTable.m

```
function T = weightingTable()
%WEIGHTINGTABLE A/B/C frequency-weighting relative response (dB).
% T = ACOUSTICS.WEIGHTINGTABLE() returns an N-by-4 matrix
% [freq_Hz, A_dB, B_dB, C_dB] covering the 1/3-octave centre
% frequencies 25 Hz - 20 kHz (IEC 61672 family).
%
% See also ACOUSTICS.WEIGHTINGVALUE.
T = [ 25 -44.7 -20.4 -4.4; 31.5 -39.4 -17.1 -3.0; 40 -34.6 -14.2 -2.0;
      50 -30.2 -11.6 -1.3; 63 -26.2 -9.3 -0.8; 80 -22.5 -7.4 -0.5;
      100 -19.1 -5.6 -0.3; 125 -16.1 -4.2 -0.2; 160 -13.4 -3.0 -0.1;
      200 -10.9 -2.0 0.0; 250 -8.6 -1.3 0.0; 315 -6.6 -0.8 0.0;
      400 -4.8 -0.5 0.0; 500 -3.2 -0.3 0.0; 630 -1.9 -0.1 0.0;
      800 -0.8 0.0 0.0; 1000 0.0 0.0 0.0; 1250 0.6 0.0 0.0;
      1600 1.0 0.0 -0.1; 2000 1.2 -0.1 -0.2; 2500 1.3 -0.2 -0.3;
      3150 1.2 -0.4 -0.5; 4000 1.0 -0.7 -0.8; 5000 0.5 -1.2 -1.3;
      6300 -0.1 -1.9 -2.0; 8000 -1.1 -2.9 -3.0; 10000 -2.5 -4.3 -4.4;
      12500 -4.3 -6.1 -6.2; 16000 -6.6 -8.4 -8.5; 20000 -9.3 -11.1 -11.2 ];
end
```

weightingValue.m

Source file: +acoustics/weightingValue.m

```
function w = weightingValue(freq, net)
%WEIGHTINGVALUE Relative response (dB) of a weighting network at a frequency.
% w = ACOUSTICS.WEIGHTINGVALUE(freq, net) looks up the A/B/C weighting
% value (dB) at the standard band centre FREQ (Hz). NET is one of
% 'A','B','C' or 'Z' (no weighting -> 0). Non-standard frequencies not
% present in the table return 0.
%
% See also ACOUSTICS.WEIGHTINGTABLE, ACOUSTICS.WEIGHTEDOVERALL.
arguments
    freq (1,1) double {mustBePositive}
    net (1,1) char {mustBeMember(net,{'A','B','C','Z'})}
end
if net == 'Z'
    w = 0; return;
end
T = acoustics.weightingTable();
col = struct('A',2,'B',3,'C',4);
row = find(abs(T(:,1) - freq) < 1e-6, 1);
if isempty(row)
    w = 0;
else
```

```
        w = T(row, col.(net));  
    end  
end
```